

**A Web-Based Learning Assistant for University Students Using Text Summarization and
Question Generation with Natural Language Processing APIs**

By

Tham Yan Wai

A PROPOSAL

SUBMITTED TO

Universiti Tunku Abdul Rahman

in partial fulfillment of the requirements

for the degree of

BACHELOR OF INFORMATION TECHNOLOGY (HONOURS)

COMMUNICATIONS AND NETWORKING

Faculty of Information and Communication Technology

(Kampar Campus)

FEBRUARY 2026

ACKNOWLEDGEMENTS

The author would like to express sincere appreciation to the project supervisor for the continuous guidance, constructive comments, and valuable feedback given throughout the preparation of this Final Year Project I report. The author is also grateful to the Faculty of Information and Communication Technology, Universiti Tunku Abdul Rahman, for providing the academic environment and facilities that supported the completion of this work. Appreciation is further extended to family members and friends for their encouragement, patience, and support during the development of the project.

COPYRIGHT STATEMENT

© 2026 Tham Yan Wai. All rights reserved.

This Final Year Project proposal is submitted in partial fulfillment of the requirements for the degree of Bachelor of Information Technology (Honours) Communications and Networking at Universiti Tunku Abdul Rahman (UTAR). This Final Year Project proposal represents the work of the author, except where due acknowledgment has been made in the text. No part of this Final Year Project proposal may be reproduced, stored, or transmitted in any form or by any means, whether electronic, mechanical, photocopying, recording, or otherwise, without the prior written permission of the author or UTAR, in accordance with UTAR's Intellectual Property Policy.

ABSTRACT

Note2Quiz was proposed as an AI-assisted study application to reduce the manual effort required to convert lecture notes and learning materials into usable revision resources. Students often spend substantial time rereading notes, identifying key ideas, and preparing their own practice questions before effective revision can begin. Existing tools usually support only isolated activities such as flashcards, quizzes, or live classroom games, rather than an integrated workflow that starts from student-owned material. This project therefore proposed a cross-platform system that could automatically transform uploaded notes into interactive revision content, including quizzes, flashcards, mind-map style summaries, briefing-style reports, and prototype live multiplayer quiz sessions. The prototype was implemented with a Flutter frontend, a Flask backend, MongoDB for notebook persistence, Firebase Authentication for sign-in, and the Gemini API for AI-assisted generation. At the FYP I stage, success was assessed through the functional operation of the core workflow: user access, notebook creation, PDF and PPTX upload, source-text extraction, AI content generation, solo quiz interaction, and multiplayer room creation and joining. The project was significant because it demonstrated a practical way to improve revision efficiency, reduce repetitive preparation work, and increase study engagement through both self-paced and classroom-oriented revision activities.

Area of Study: Educational Technology; Artificial Intelligence

Keywords: AI-assisted learning, revision support, quiz generation, flashcards, note processing, multiplayer quiz

TABLE OF CONTENTS

TITLE PAGE	i
ACKNOWLEDGEMENTS	ii
COPYRIGHT STATEMENT	iii
ABSTRACT	iv
TABLE OF CONTENTS	v
LIST OF FIGURES	viii
LIST OF TABLES	x
LIST OF SYMBOLS	xi
LIST OF ABBREVIATIONS	xii
CHAPTER 1 INTRODUCTION	1
1.1 Project backgroundd	1
1.2 Problem Statement and Motivation	2
1.3 Project Objectives	3
1.4 Project Contributions	3
1.5 Project Scope	4
1.6 Report Organization	5

CHAPTER 2 LITERATURE REVIEW	6
2.1 Introduction	6
2.2 Review of Technologies	6
2.2.1 Field of Study: AI-Assisted Digital Revision	6
2.2.2 Application Development Frameworks	7
2.2.3 Backend Frameworks	9
2.2.4 Database Systems	11
2.2.5 AI-Assisted Content Generation Approaches	13
2.3 Review of Existing Systems/Applications	14
2.4 Chapter Summary	16
 CHAPTER 3 PROPOSED METHOD/APPROACH	 17
3.1 Introduction to System Development Models	17
3.1.1 Waterfall Model	17
3.1.2 Agile Model	18
3.1.3 Prototype Model	18
3.1.4 Selected Model	19
3.2 System Requirements	20
3.2.1 Hardware Components	20
3.2.2 Software Components	21
3.3 System Design	23
3.3.1 System Architecture	23
3.3.2 Module Design	24
3.3.3 Data Flow Overview	25
3.4 Functional Requirements	27
3.5 Project Milestone	29
3.5.1 Project I Timeline	29
3.5.2 Project II Timeline	29
3.6 Concluding Remark	30

CHAPTER 1

CHAPTER 4 PRELIMINARY WORK 31

4.1 Software Setup and Configuration	31
4.1.1 Frontend Development and Environment	31
4.1.2 Firebase Authentication and Configuration	32
4.1.3 Backend Environments and Require Libraries	32
4.1.4 Database Configuration	33
4.1.5 Front-end-to-Backend Address Configuration	34
4.2 Implementation details	35
4.2.1 Access and Login Module	35
4.2.2 Dashboard and Notebook Management Module	36
4.2.3 Source Upload and Text Extraction Module	36
4.2.4 AI Content Generation Module	37
4.2.5 Solo Quiz and Flashcard Modules	38
4.2.6 Multiplayer Game Module	38
4.3 System Operation	39
4.3.1 Design Layout	39
4.3.2 Functional Operation	43
4.4 Implementation Issues and Challenges	48
4.5 Concluding Remark	49

CHAPTER 5 CONCLUSION 50

REFERENCES 52

APPENDIX A

A.1 Questionnaire Sample	A-1
A.2 Code Sample	A-2
A.3 Poster	A-4

LIST OF FIGURES

Figure 3.1.1 Waterfall model diagram	17
Figure 3.1.2 Agile model diagram	18
Figure 3.1.3 Prototype model diagram	19
Figure 3.2.1 Visual Studio Code development environment.....	22
Figure 3.2.2 Firebase authentication configuration dashboard	22
Figure 3.2.3 Flask backend runtime environment	23
Figure 3.2.4 MongoDB database environment	23
Figure 3.3.1 Overall system architecture of Note2Quiz	24
Figure 3.3.2 Top-down module interaction diagram	25
Figure 3.3.3 Data flow from notebook source upload to AI-generated study output	27
Figure 3.5.1 Project I milestone chart.....	29
Figure 3.5.2 Project II milestone chart.....	30
Figure 4.1.1 Visual Studio Code with the Note2Quiz project opened.....	31
Figure 4.1.2 Firebase project dashboard used for web authentication setup	32
Figure 4.1.3 Flask backend running in the terminal during local development	33
Figure 4.1.4 MongoDB collections used for notebooks, users, and activity logs.....	34
Figure 4.1.5 Base URL configuration for web and emulator environments.....	35
Figure 4.3.1 Landing page design.....	40
Figure 4.3.2 Login page design.....	40
Figure 4.3.3 Dashboard design	41
Figure 4.3.4 Notebook workspace design.....	41
Figure 4.3.5 Solo quiz design	42
Figure 4.3.6 Student lobby design	42
Figure 4.3.7 Teacher leaderboard design.....	43
Figure 4.3.8 Notebook creation dialog	43
Figure 4.3.9 Source upload into a notebook	44
Figure 4.3.10 Generated briefing-style report output	44
Figure 4.3.11 Generated mind-map style summary output.....	45
Figure 4.3.12 Quiz-ready state after generation.....	45

Figure 4.3.13 Solo quiz interaction screen.....	46
Figure 4.3.14 Flashcard interaction view.....	46
Figure 4.3.15 Join live game dialog.....	47
Figure 4.3.16 Teacher leaderboard with room code and start control	47
Figure 4.3.17 Multiplayer quiz gameplay screen.....	48

LIST OF TABLES

Table 2.1 Comparison of frontend development approaches	7
Table 2.2 Comparison of backend framework options	9
Table 2.3 Comparison of database options	11
Table 2.4 Comparison of content generation approaches	13
Table 2.5 Comparison between existing systems and the proposed system.....	16
Table 3.1 Hardware requirements for the prototype environment.....	20
Table 3.2 Software requirements for the prototype environment	21
Table 3.3 Functional requirements of Note2Quiz (Part 1)	27
Table 3.4 Functional requirements of Note2Quiz (Part 2)	28
Table 3.5 Functional requirements of Note2Quiz (Part 3)	28

LIST OF SYMBOLS

No specialised mathematical symbols were used in this report.

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
FYP	Final Year Project
JSON	JavaScript Object Notation
PDF	Portable Document Format
PPTX	PowerPoint Open XML Presentation
REST	Representational State Transfer
UI	User Interface
UTAR	Universiti Tunku Abdul Rahman

CHAPTER 1

Introduction

1.1 Project Background

The widespread use of digital lecture slides, notes, and online learning material has changed how students access knowledge, but it has not eliminated the effort required to prepare for revision. Effective study usually depends on active recall and retrieval practice rather than repeated passive reading alone [1], [2]. In practice, however, many students still revise by manually rereading notes, highlighting slides, summarizing key ideas, and writing their own practice questions. This preparation stage is often lengthy and repetitive, especially near examination periods, and the time spent organizing revision materials can reduce the time available for actual practice. As a result, revision efficiency may decrease and student motivation may also be affected.

Digital technologies offer opportunities to address these challenges more effectively. Cross-platform application frameworks make it possible to provide a single study environment across web and mobile devices, while cloud authentication and database services support consistent user access and notebook storage. More importantly, recent advances in generative AI make it possible to automatically convert uploaded material into structured learning resources such as quizzes, flashcards, summaries, and concept maps [5], [6]. Compared with manual preparation, such an approach can reduce repetitive work, speed up revision preparation, and provide a more consistent path from source material to practice activity.

This project therefore proposes Note2Quiz, an AI-powered cross-platform study application that allows students to organize notebooks, upload academic material, and automatically generate interactive revision content from those materials. The project is significant because it aims to reduce manual preparation effort, improve study efficiency, and support both independent and collaborative revision. In the longer term, the system is expected to provide a more engaging and practical revision workflow for students who already rely on digital learning resources.

1.2 Problem Statement and Motivation

Revision is a critical part of learning because students must not only read material but also practice recalling and applying it. Despite the growth of digital learning environments, students often move between lecture slides, personal notes, messaging tools, and separate quiz applications during revision. This fragmented process creates a gap between the availability of digital content and the availability of structured revision support. The present project is motivated by this gap and focuses on how study material can be converted into interactive revision content more directly and more efficiently.

The first problem is the amount of time required to manually transform source material into revision resources. Students commonly need to identify important concepts, rewrite summaries, and prepare self-test questions before meaningful practice can start. When this conversion process is slow, students may spend more time preparing to revise than actually revising. The consequence is reduced study efficiency, weaker revision consistency across topics, and potentially lower confidence before tests or examinations.

The second problem is fragmentation across existing study tools. Many applications support only a single revision activity, such as flashcards, quizzes, or source summarization, without offering a complete flow from uploaded notes to practice. When students must switch repeatedly between platforms, they face additional cognitive load, repeated copy-and-paste work, and the risk of losing context between preparation and practice. This fragmentation can discourage regular revision and makes the study process less convenient than it should be.

The third problem is limited engagement in traditional self-study workflows. Revision that depends only on reading and isolated self-testing may become monotonous, especially when students study alone. Reduced engagement can affect motivation, participation, and sustained practice over time. In classroom or group settings, the absence of lightweight live quiz interaction also reduces opportunities for collaborative revision. A study system that combines automatic content generation with solo and multiplayer activities can therefore make revision more engaging as well as more efficient.

In summary, manual preparation effort, fragmented tools, and low-engagement revision workflows remain important barriers to effective study. These problems create a clear need for a system that can automatically convert uploaded notes into interactive revision content within one

environment. The project is motivated by the expectation that such a system can improve convenience, support active recall, and create a more engaging revision experience for both individual learners and small classroom groups.

1.3 Project Objectives

Based on the background and problems identified above, the project establishes three specific objectives that are aligned with the aim of reducing manual revision effort and improving study engagement. The objectives are stated as concrete outputs for the FYP 1 and FYP 2 development period.

- To develop a cross-platform study application that supports user access, notebook creation, and the upload of PDF and PPTX study sources into organized notebook workspaces.
- To implement automatic AI-based generation of interactive revision content, namely quizzes, flashcards, mind-map style summaries, and briefing-style reports, using notebook source context.
- To implement and functionally verify solo quiz practice and prototype multiplayer quiz flows so that generated quiz content can be used for both self-revision and classroom-style activities.

The first objective is concerned with the core study workspace. A working prototype should demonstrate that users can enter the application, create notebooks, and upload supported study material successfully. This objective is measurable through the correct operation of the relevant frontend screens and backend endpoints.

The second objective focuses on automatic content generation. The project does not treat AI as a replacement for learning; instead, AI is used to transform source material into structured revision content. This objective is measurable through successful generation requests and the display of usable quiz, flashcard, and summary-style outputs from uploaded notebook content.

The third objective addresses the revision experience itself. The generated quiz content should not remain static; it should support actual learning interaction through solo practice and prototype multiplayer use. This objective is measurable through working quiz flows, score tracking, room creation, room joining, and leaderboard updates within the project scope.

1.4 Project Contribution

The proposed project contributes to the study technology domain by connecting notebook-based source organization with automatic revision support in a single application. Instead of separating source storage, content generation, and practice into unrelated tools, the system combines them into one workflow that begins with student-owned material.

The first contribution is the reduction of manual preparation effort. By automatically generating revision content from uploaded notes, the system reduces repetitive tasks such as rewriting key points or manually composing practice questions. This contribution directly addresses the first problem statement and supports faster preparation for retrieval practice.

The second contribution is workflow integration. Note2Quiz combines user access, notebook organization, source upload, AI generation, and revision activities inside one environment. This integrated structure reduces fragmentation, lowers switching overhead, and makes it easier for students to move from raw material to usable revision output.

The third contribution is improved engagement through interactive revision. The project provides both solo quiz practice and a lightweight multiplayer quiz flow with a room code and leaderboard. This contribution supports more engaging study sessions and introduces a classroom-friendly revision option that extends beyond passive reading.

Overall, the project contributes a practical prototype for AI-assisted revision that is grounded in student-owned source material, reduces repetitive preparation work, and supports both individual and collaborative study activities.

1.5 Project Scope

The project scope defines the boundaries of what is included in the current prototype and what is intentionally excluded so that the work remains realistic within the FYP timeline.

Inclusions:

- Support user entry through Google sign-in, email-based access, and guest access.
- Provide notebook creation, retrieval, and deletion functions linked to a user identifier.
- Allow source upload in PDF and PPTX formats through the current user interface.

CHAPTER 1

- Extract text from uploaded study material and store it as notebook context for later AI processing.
- Generate quizzes, flashcards, mind-map style summaries, and briefing-style reports from notebook content.
- Provide solo quiz interaction with question progression, answer checking, and score display.
- Provide prototype multiplayer hosting, joining, start control, score updates, and leaderboard display using a generated room code.

Exclusions:

- The current interface does not include a complete YouTube transcript upload workflow, even though a transcript library is imported on the backend.
- The multiplayer prototype does not provide production-scale real-time synchronization or persistence across backend restarts because active game state is stored in server memory.
- Advanced analytics, adaptive learning, offline synchronization, large-scale classroom management, and cloud deployment are outside the current project scope.
- Rich document editing, export formatting, and production-grade security hardening are not included at the present stage.

In summary, the project focuses on a functional AI-assisted study prototype with core notebook, generation, solo quiz, and prototype multiplayer features. Advanced deployment, analytics, and large-scale operational features are intentionally excluded to keep the project aligned with the FYP 1 scope and timeline.

1.6 Report Organization

Chapter 1 introduces the project background, problem statement, objectives, contribution, and scope. Chapter 2 reviews relevant technologies and existing study applications. Chapter 3 presents the development methodology, requirements, system design, functional requirements, and project milestones. Chapter 4 describes the preliminary work, including software setup, implementation details, system operation, and implementation challenges. Chapter 5 concludes the FYP 1 stage and outlines the current prototype status together with the direction for subsequent work.

CHAPTER 2

Literature Review

2.1 Introduction

This chapter reviews the learning concepts, technologies, and related systems that form the foundation of Note2Quiz. Because the project combines AI-assisted study generation, cross-platform application development, cloud-supported authentication, and live quiz interaction, the literature review must consider both educational needs and enabling technologies. The review also examines similar applications in order to identify the gap addressed by the proposed system.

2.2 Review of Technologies

2.2.1 Field of Study: AI-Assisted Digital Revision

The project is situated within the broader field of digital learning support. One of the strongest theoretical foundations for this field is retrieval practice, which shows that testing and self-recall can improve long-term retention more effectively than repeated passive study alone [1], [2]. For this reason, a revision system that helps students generate quiz-style practice from their own notes is pedagogically relevant rather than merely convenient.

The field is also influenced by gamification and interactive learning design. Reviews of educational gamification report that score-based activities, competition, and structured feedback can improve engagement and participation when used appropriately [3], [4]. These findings support the inclusion of solo scoring and lightweight leaderboard-based multiplayer activities in the proposed system.

Generative AI has recently become important in education because it can assist with summarization, content generation, and personalized study support [5], [6]. However, the most useful role for generative AI in this project is not to replace student learning but to reduce repetitive preparation work. By helping students convert their own notes into structured revision content, AI can support rather than substitute the learning process.

2.2.2 Application Development Frameworks

2.2.2.1 Native Mobile Development

Native development targets a specific operating system, such as Android or iOS, using the platform's own tools and software development kit [9], [10]. Its main strengths are strong performance, direct access to platform capabilities, and a user experience that closely follows platform conventions. Its main weakness for a student project is the need to maintain separate codebases or platform-specific implementation effort, which increases development time and maintenance overhead.

2.2.2.2 React Native

React Native is a cross-platform framework that allows mobile applications to be developed with JavaScript and React concepts [8]. It supports rapid interface development and code reuse across platforms. Its strengths include a large ecosystem and a familiar model for web-oriented developers. However, integration complexity can increase when a project depends on multiple plugins, and user interfaces may require additional bridging for platform-specific behavior.

2.2.2.3 Flutter

Flutter is a cross-platform user interface toolkit that supports a single codebase for web, mobile, and desktop-style deployment [7]. Its strengths include strong widget consistency, fast interface development, and good suitability for UI-heavy projects. For Note2Quiz, Flutter is especially appropriate because the application requires multiple study screens, notebook workspaces, quiz views, and live game interfaces that must remain visually consistent across environments.

2.2.2.4 Summary

Table 2.1 Comparison of frontend development approaches

Framework / approach	Strengths	Weaknesses	Suitability for Note2Quiz
Native mobile development	Strong performance, direct platform integration	Separate platform effort and higher maintenance cost	Less suitable because the project targets rapid cross-platform prototyping
React Native	Cross-platform reuse, large ecosystem, rapid UI development	Plugin dependency complexity and extra bridging in some	Moderately suitable but less aligned with the selected Flutter

		cases	stack
Flutter	Single codebase, consistent UI, strong widget system	Some platform-specific setup is still required	Most suitable because the project is interface-heavy and cross-platform in scope

In summary, Flutter is selected as the frontend framework because it provides an effective balance between cross-platform compatibility, consistent user interface design, and rapid development, all of which are essential for the Note2Quiz system. The application includes multiple interconnected modules such as login, dashboard, notebook workspace, quiz interfaces, flashcards, and multiplayer screens. Flutter's widget-based architecture allows these interfaces to be developed using a unified design approach, ensuring a consistent user experience across different screens and platforms.

One key reason for choosing Flutter is its ability to support cross-platform deployment from a single codebase. Since Note2Quiz is designed to run in both web and mobile-like environments, Flutter enables efficient development without the need to maintain separate implementations for each platform. This significantly reduces development time and complexity, which is important for a Final Year Project with limited duration.

In addition, Flutter supports rapid prototyping through features such as hot reload, which allows real-time updates to the user interface during development. This is particularly beneficial for this project, as the system follows a Prototype Model where features such as the notebook layout, quiz interaction flow, and multiplayer interfaces were refined iteratively. Flutter enables quick adjustments and immediate visual feedback, supporting continuous improvement of the user interface.

Furthermore, Flutter integrates well with backend services through HTTP-based communication, which aligns with the system's architecture where the frontend interacts with a Flask REST API. This allows smooth data exchange for operations such as user login, notebook retrieval, file upload, AI content generation, and multiplayer game updates. The ability to handle asynchronous requests and dynamic UI updates makes Flutter suitable for interactive features such as quiz progression and live leaderboard updates.

Although Flutter may require additional configuration for certain platform-specific integrations, its advantages in development speed, UI consistency, and cross-platform support outweigh these limitations. Therefore, Flutter is the most suitable choice for implementing the frontend of the Note2Quiz system.

2.2.3 Backend Frameworks

2.2.3.1 Flask

Flask is a lightweight Python web framework that is commonly used to build application programming interfaces and web services [11]. For this project, Flask is attractive because it is simple to structure, compatible with Python document-parsing and AI libraries, and well suited to Representational State Transfer (REST) API development. Its main weakness is that it provides less built-in structure than heavier frameworks.

2.2.3.2 Django

Django is a more full-featured Python framework that includes a stronger built-in project structure, administration tools, and integrated conventions [12]. These features are useful for larger systems, but they can be unnecessarily heavy for a prototype that mainly requires a clean API layer and flexible integration with external services.

2.2.3.3 Node.js / Express

Node.js with Express is another common option for lightweight API development. Express provides routing and middleware support for request handling [13]. It is flexible and popular, but it does not provide a direct advantage for this project because the application's document parsing and AI integration pipeline is already based in Python.

2.2.3.4 Summary and Justification

Table 2.2 Comparison of backend framework options

Backend option	Strengths	Weaknesses	Suitability for Note2Quiz
Flask	Simple REST API development, strong Python integration	Less built-in structure for large systems	Most suitable because the backend depends on Python parsing and

			AI libraries
Django	Comprehensive framework and built-in tools	Heavier than required for a focused prototype API	Suitable but less efficient for the selected architecture
Node.js / Express	Flexible routing and middleware, large community	Would require a separate ecosystem from the chosen Python AI pipeline	Moderately suitable but not the best fit for the implemented stack

Flask is selected as the backend framework because it provides a lightweight yet highly flexible environment for building RESTful APIs that align closely with the requirements of the Note2Quiz system. Unlike full-stack frameworks such as Django, Flask follows a minimalistic design philosophy, allowing developers to implement only the necessary components without unnecessary overhead. This is particularly suitable for this project, where the backend primarily functions as an API layer that handles notebook management, file uploads, AI content generation requests, and multiplayer game coordination.

One key reason for choosing Flask is its strong compatibility with Python-based libraries and services. The system relies heavily on Python tools such as PDF parsing (pypdf), PowerPoint processing (python-pptx), and AI integration using the Gemini API. Flask allows these libraries to be integrated directly and efficiently within the same environment, simplifying development and reducing the need for additional middleware. This direct integration is evident in the backend implementation, where endpoints handle tasks such as text extraction, AI prompt generation, and response processing within a unified Python workflow.

In addition, Flask supports rapid development and iterative prototyping, which aligns well with the selected Prototype Model used in this project. Features such as quiz generation, flashcard generation, and multiplayer sessions were developed and refined incrementally. Flask's simple routing system and modular structure make it easy to add, modify, and test endpoints during development, enabling faster experimentation and continuous improvement.

Another important advantage is Flask's suitability for handling RESTful communication between the Flutter frontend and backend services. The system architecture depends on clear API endpoints (e.g., login, notebook creation, source upload, AI generation, and game management), and Flask provides a straightforward way to define these routes and return JSON responses. This ensures smooth communication between the client and server components.

Although Flask does not provide built-in features for large-scale application structure or scalability, this limitation is acceptable for a Final Year Project prototype. The project focuses on functionality, modular design, and integration of AI services rather than enterprise-level scalability. Therefore, Flask offers an optimal balance between simplicity, flexibility, and development efficiency for the scope of this system.

2.2.4 Database Systems

2.2.4.1 MongoDB

MongoDB is a document-oriented database designed for application development and flexible schema design [14]. It is suitable for Note2Quiz because notebook records naturally contain nested data such as notebook metadata, source lists, and generated content context. This flexibility supports the evolving structure of notebook documents during prototyping.

2.2.4.2 MySQL

MySQL is a widely used relational database system [15]. It offers strong structure, mature tooling, and well-defined relational modelling. However, the notebook data used in this project is more naturally expressed as nested documents than as multiple related tables, so MySQL would require a more rigid schema for a comparatively flexible prototype.

2.2.4.3 Cloud Firestore

Cloud Firestore is a flexible NoSQL cloud database that is tightly integrated with Firebase [16]. It offers good synchronization support, but for this project MongoDB already serves the required notebook persistence role on the backend, while Firebase is used mainly for authentication. Using Firestore as an additional main database would duplicate responsibilities in the current architecture.

2.2.4.4 Summary and Justification

Table 2.3 Comparison of database options

Database	Strengths	Weaknesses	Suitability for Note2Quiz
MongoDB	Flexible document model, natural fit for nested notebook data	Schema discipline must be managed carefully	Most suitable for notebook and source records
MySQL	Strong relational structure and mature	Less natural for nested notebook documents in	Suitable for rigid relational systems but

	tooling	this prototype	not the best fit here
Cloud Firestore	Cloud-based NoSQL service with Firebase integration	Would duplicate database responsibilities in the current design	Partially suitable but not selected as the main notebook database

In summary, MongoDB is selected as the database system because it provides a flexible and scalable data storage solution that aligns well with the structure and requirements of the Note2Quiz application. The system manages data such as user accounts, notebooks, uploaded sources, and generated study content, all of which have dynamic and nested structures. A document-oriented database like MongoDB is well suited for this type of data, as it allows information to be stored in JSON-like documents without requiring a fixed schema.

One key reason for choosing MongoDB is its ability to handle variable and hierarchical data structures efficiently. In Note2Quiz, each notebook may contain multiple sources, and each source includes fields such as title, extracted content, type, and timestamp. This nested structure can be stored naturally within a single document, reducing the need for complex table relationships and joins that would be required in relational databases. This simplifies both data design and query operations.

In addition, MongoDB supports rapid development and iteration, which is important for a project following the Prototype Model. As the system evolves, new fields or features—such as activity logs or additional metadata—can be added without requiring major changes to the database schema. This flexibility allows the developer to refine the data model continuously as new functionalities are introduced.

Another advantage is MongoDB's compatibility with the Flask backend through libraries such as PyMongo, enabling straightforward integration between the API layer and the database. This supports efficient handling of operations such as notebook creation, source storage, and activity logging within the backend workflow.

Although MongoDB requires careful data validation to avoid inconsistencies due to its schema-less nature, this limitation is manageable within the scope of this project. Therefore, MongoDB provides an effective balance between flexibility, simplicity, and development efficiency, making it a suitable choice for the Note2Quiz system.

2.2.5 AI-Assisted Content Generation Approaches

2.2.5.1 Rule-Based Generation

A rule-based generation approach can only create study output when explicit templates or heuristics have been defined in advance. Its main strength is predictability, but it is limited when the source material varies significantly in style or subject matter.

2.2.5.2 Template-Based Generation

A template-only approach provides more formatting control than pure rule-based generation, but it still depends on manually defined structures and may not adapt well to unstructured lecture notes. This reduces its usefulness for a system that must process different note formats with minimal manual intervention.

2.2.5.3 Generative AI Approach

Generative AI provides greater flexibility because it can interpret source context and produce multiple output types from the same material [17]. In Note2Quiz, this allows a single notebook to generate quiz questions, flashcards, mind-map style summaries, and briefing-style reports. The main limitations are response consistency, dependence on prompt design, and the need for output validation.

2.2.5.4 Summary and Justification

Table 2.4 Comparison of content generation approaches

Approach	Strengths	Weaknesses	Suitability for Note2Quiz
Rule-based generation	Predictable output	Low flexibility with varied source content	Insufficient for broad note-to-study conversion
Template-only generation	Good formatting control	Still limited by manual patterns and fixed structures	Useful as support logic but not as the main generator
Generative AI (Gemini API)	Flexible multi-output generation from unstructured context	Needs prompt design and response cleaning	Most suitable for the current project goal

Among the approaches discussed, the generative AI approach is selected for this project because it best supports the transformation of unstructured and heterogeneous student learning

materials into multiple forms of revision content. In Note2Quiz, users are allowed to upload different types of documents such as PDF lecture notes and PowerPoint slides, which often vary significantly in structure, formatting, and level of detail. Rule-based and template-based approaches require predefined patterns or fixed structures, making them unsuitable for handling such variability without extensive manual configuration.

In contrast, generative AI can interpret contextual information from extracted text and dynamically generate meaningful outputs such as quizzes, flashcards, mind maps, and briefing-style summaries. This capability enables the system to reuse a single notebook context to produce multiple types of study materials without requiring separate rule definitions for each output format. As a result, it significantly reduces development complexity while improving scalability for different subjects and content styles.

Furthermore, the use of generative AI aligns with the system's design objective of minimizing user effort. Instead of requiring users to manually structure their notes or follow strict input formats, the system can process raw text directly and generate usable revision content. This enhances usability and makes the system more accessible to students with different study habits.

Although generative AI introduces challenges such as response inconsistency, dependence on prompt design, and the need for output validation, these limitations are addressed in the system through structured prompting strategies and post-processing mechanisms. Therefore, despite its limitations, generative AI provides the most suitable balance between flexibility, functionality, and implementation feasibility for the Note2Quiz system.

2.3 Review of Existing Systems / Applications

2.3.1 Quizlet

Quizlet is a widely used study platform known for flashcards, practice tests, and AI-supported study features [18]. Its strengths include strong support for memorization-oriented self-study and a familiar student workflow. However, it is not primarily built around a notebook-to-live-classroom path from uploaded personal notes into multiplayer revision.

2.3.2 Kahoot!

Kahoot! is strongly associated with game-based classroom learning and code-based live participation [19]. Its main strength is fast setup for competitive quiz sessions. However, it focuses on the live game experience rather than on automatically converting a student's uploaded source material into multiple study outputs inside a notebook workspace.

2.3.3 Quizizz

Quizizz supports live and assigned quiz sessions and provides multiple assessment-oriented delivery modes [20]. It is useful for classroom quiz interaction, but its design is centered more on assessment and reporting than on a source-grounded notebook workflow that begins with uploaded notes and expands into summaries, flashcards, quizzes, and multiplayer use.

2.3.4 NotebookLM

NotebookLM is a source-grounded research and learning assistant that allows users to upload documents and generate AI-assisted outputs from those sources [21]. This makes it particularly relevant to Note2Quiz because both systems begin with user-provided material. Its strength lies in source understanding and grounded responses. However, it does not primarily target lightweight live classroom quiz competition in the way Note2Quiz does.

2.3.5 Proposed System and Summary

The review above indicates that existing systems tend to focus on specific segments of the learning process rather than providing a fully integrated workflow. Quizlet primarily supports self-study through flashcards and practice tests, Kahoot! emphasizes real-time classroom engagement through live quiz sessions, while Quizizz focuses on structured quiz delivery and assessment features. NotebookLM, on the other hand, excels in source-grounded AI assistance by enabling users to analyze and summarize uploaded materials. However, each of these systems operates within a limited scope and does not fully support the end-to-end transformation of student-owned learning materials into multiple interactive revision formats within a single environment.

The key gap identified is the lack of an integrated system that connects all stages of the revision process. Specifically, existing platforms do not provide a seamless workflow that begins with notebook-based source upload, followed by automated generation of diverse study content,

CHAPTER 2

and extends to both self-paced revision and interactive multiplayer engagement. This fragmentation requires users to rely on multiple tools, which increases cognitive load and reduces overall efficiency.

Note2Quiz addresses this gap by combining these functionalities into a unified application. It enables users to upload personal study materials into structured notebooks, automatically generate quizzes, flashcards, mind maps, and briefing-style summaries, and then utilize these outputs in both individual and multiplayer revision modes. By integrating content generation and interaction within a single system, Note2Quiz offers a more coherent and efficient learning workflow that improves accessibility, reduces manual effort, and enhances user engagement.

Table 2.5 Comparison between existing systems and the proposed system

Feature	Quizlet	Kahoot!	Quizizz	NotebookLM	Note2Quiz
Flashcards	Yes	Limited	Limited	Partial	Yes
Live multiplayer quiz	Limited	Yes	Yes	No	Yes
Upload personal notes / slides	Yes	Limited	Limited	Yes	Yes
AI-based study content generation	Yes	Partial	Partial	Yes	Yes
Notebook workspace	Limited	No	No	Yes	Yes
Integrated path from note upload to live game	No	No	No	No	Yes

2.4 Chapter Summary

This chapter reviewed the educational motivations, technologies, and existing systems relevant to Note2Quiz. Retrieval practice, gamification, and AI-assisted learning support provide the conceptual basis for the proposed system. Flutter, Flask, MongoDB, Firebase Authentication, and the Gemini API were reviewed as enabling technologies, while Quizlet, Kahoot!, Quizizz, and NotebookLM were examined to identify the functional gap addressed by the project. The review shows that there is room for a system that connects uploaded source material directly to multiple interactive revision activities in one workflow.

CHAPTER 3

Proposed Method/Approach

3.1 Introduction to System Development Models

System development models provide a structured approach for planning, building, refining, and evaluating software systems. For a project such as Note2Quiz, the selected model had to support interface refinement, evolving requirements, and repeated testing of user workflows. This section therefore compared three common development models before identifying the model that best fitted the nature of the proposed system [23], [24].

3.1.1 Waterfall Model

The Waterfall model organizes development into a linear sequence of stages, typically moving from requirements to design, implementation, testing, deployment, and maintenance [23], [24]. Its main advantage is clarity because each phase has a defined output and documentation boundary. However, the model is less flexible when user interface decisions or functional requirements change during development.

Figure 3.1.1 illustrates the step-by-step nature of the Waterfall model. For Note2Quiz, this approach is less suitable because important interface and workflow decisions only became clearer after early implementation and testing.

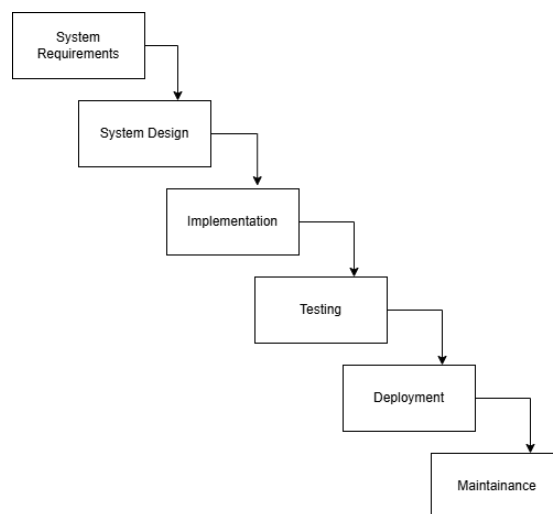


Figure 3.1.1 Waterfall model diagram

3.1.2 Agile Model

The Agile model emphasizes short iterations, continuous feedback, and incremental improvement [23], [24]. It is strong when requirements evolve and regular feedback is available. Its main strengths are adaptability and frequent refinement. However, for a single-developer FYP project, full Agile process overhead may be unnecessary if the project needs a simpler but still iterative approach.

Figure 3.1.2 shows the cyclical nature of Agile development. The model is more suitable than Waterfall for Note2Quiz because it supports change, but it is still broader than what is required for a small prototype-focused project.

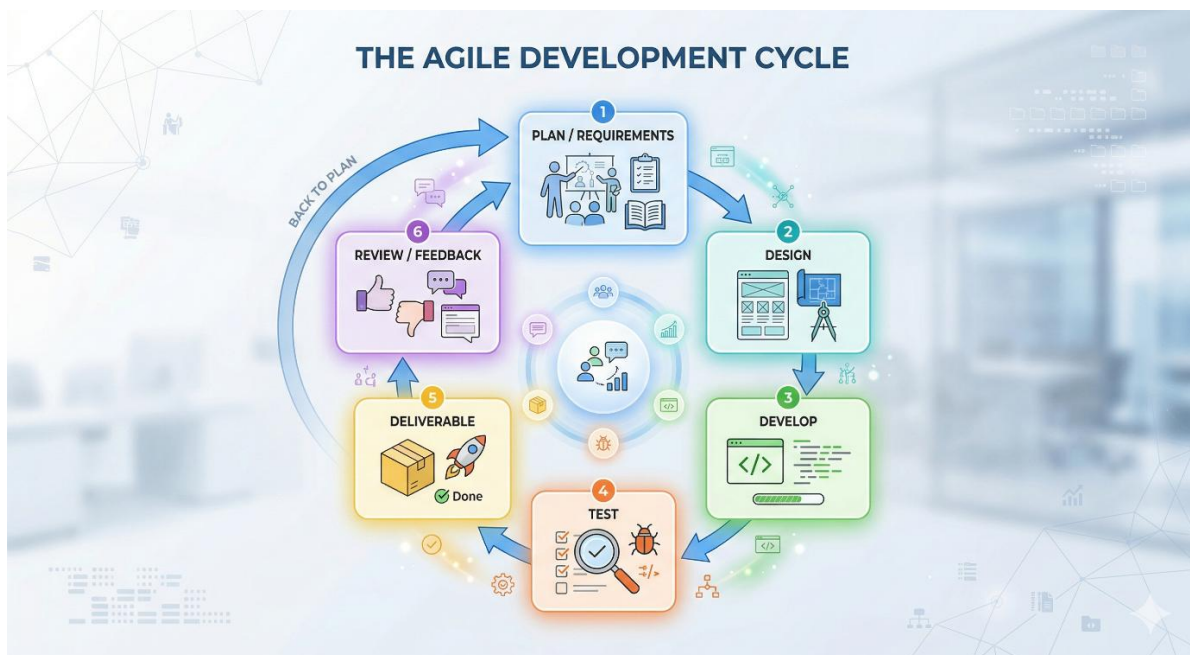


Figure 3.1.2 Agile model diagram

3.1.3 Prototype Model

The Prototype model focuses on building an early version of the system, gathering feedback from its behaviour and appearance, and refining the system through successive revisions [23], [24]. Its strengths include early validation of workflow ideas, better support for interface-heavy systems, and improved visibility of user interaction issues. Its main weakness is that the prototype can grow without discipline if revisions are not managed carefully.

As shown in Figure 3.1.3, the Prototype model relies on repeated refinement before the final system stabilizes. This is appropriate for Note2Quiz because the dashboard, notebook workspace, generation panel, and multiplayer flow benefited from progressive interface and feature refinement.

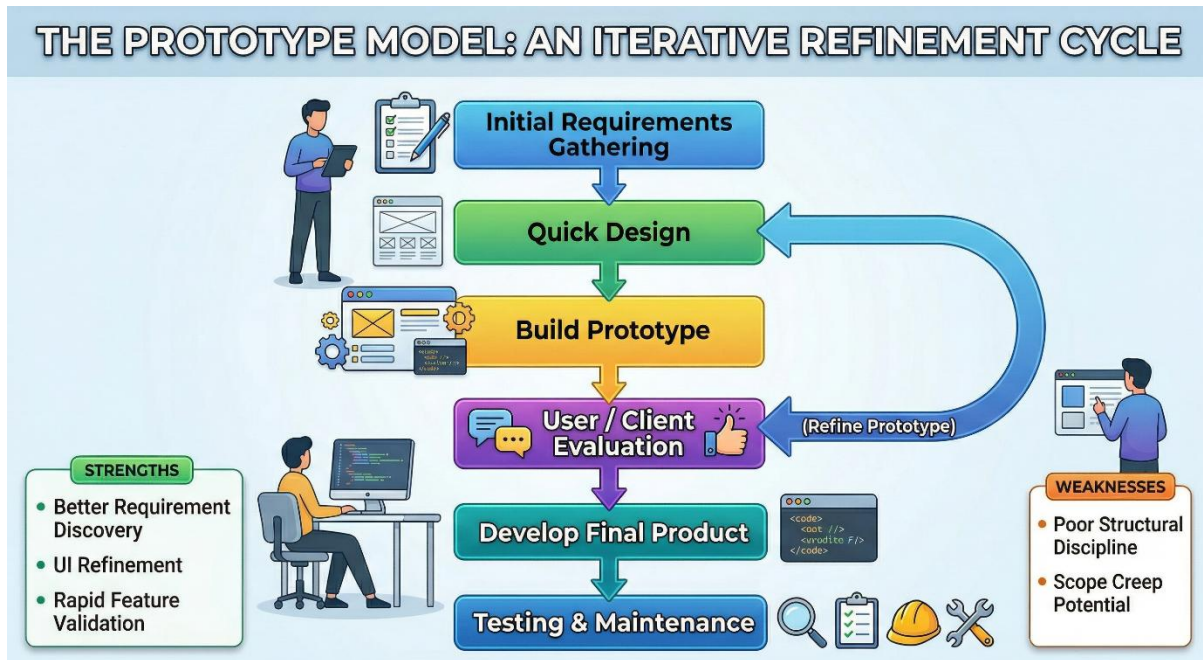


Figure 3.1.3 Prototype model diagram

3.1.4 Selected Model

The Prototype Model was selected for this project as it best aligns with the development characteristics of Note2Quiz. Compared with the Waterfall Model, the Prototype Model is more suitable because the system requires continuous refinement of user interfaces and interaction flows. In Note2Quiz, features such as the dashboard layout, notebook workspace, quiz interface, and multiplayer screens evolved progressively during development. A rigid, phase-based approach like Waterfall would limit the ability to incorporate changes once implementation has begun, making it less suitable for a system with evolving requirements.

Compared with a full Agile process, the Prototype Model provides a simpler and more manageable structure for an individual Final Year Project. Agile methodologies typically require frequent team collaboration, sprint planning, and continuous stakeholder feedback, which may not be practical in a single-developer context. In contrast, the Prototype Model allows iterative

development without the overhead of formal Agile practices, while still supporting incremental improvement.

Furthermore, the selected model reflects how the system was actually developed in practice. The project began with a basic functional prototype that included core features such as notebook creation and source upload. Subsequent iterations introduced AI content generation, followed by solo quiz interaction and multiplayer functionality. Each stage involved testing, refinement, and enhancement of both functionality and user experience, gradually transforming the system into a more complete and cohesive application.

Therefore, the Prototype Model is the most appropriate choice, as it supports iterative design, accommodates changing requirements, and aligns closely with the actual development workflow of the Note2Quiz system.

3.2 System Requirements

The successful development of Note2Quiz depends on suitable hardware and software resources. The requirements below describe the environment needed to develop, run, and test the prototype effectively.

3.2.1 Hardware Components

Table 3.1 Hardware requirements for the prototype environment

Component	Minimum requirement	Purpose
Development laptop / PC	Intel Core i5 or equivalent, 8 GB RAM, 20 GB free storage	Runs the editor, Flutter toolchain, Python backend, local database tools, and browser-based testing.
Internet connection	Stable broadband connection of at least 10 Mbps	Required for Firebase authentication workflows and Gemini API access during development and testing.
Display device	13-inch laptop screen or larger / external monitor	Supports readable interface design and debugging of multiple windows during development.
Test environment	Latest Chrome browser and Android emulator (or equivalent browser-capable device)	Supports cross-platform verification of the Flutter web interface and emulator-compatible networking.

3.2.2 Software Components

Table 3.2 Software requirements for the prototype environment

Software	Version / type	Purpose
Visual Studio Code	IDE	Main editor and debugger for frontend and backend source code.
Flutter SDK	Cross-platform framework	Frontend development and web deployment.
Dart SDK	Language runtime for Flutter	Programming environment for frontend logic.
Python	Runtime environment	Backend processing and API development.
Flask	Web framework	REST API for notebook, generation, and multiplayer requests.
MongoDB	Document database	Stores users, notebooks, and activity logs.
Firebase Authentication	Cloud authentication service	Supports Google sign-in integration on the frontend.
Gemini API	Generative AI service	Generates quizzes, flashcards, summaries, and briefing-style reports.
Google Chrome / browser runtime	Testing environment	Runs the Flutter web build for local testing and screenshots.

Representative screenshots of the principal software components used in the development environment are provided in Figures 3.2.1 to 3.2.4. These figures complement the requirement tables by showing the actual tools and services configured for the prototype.

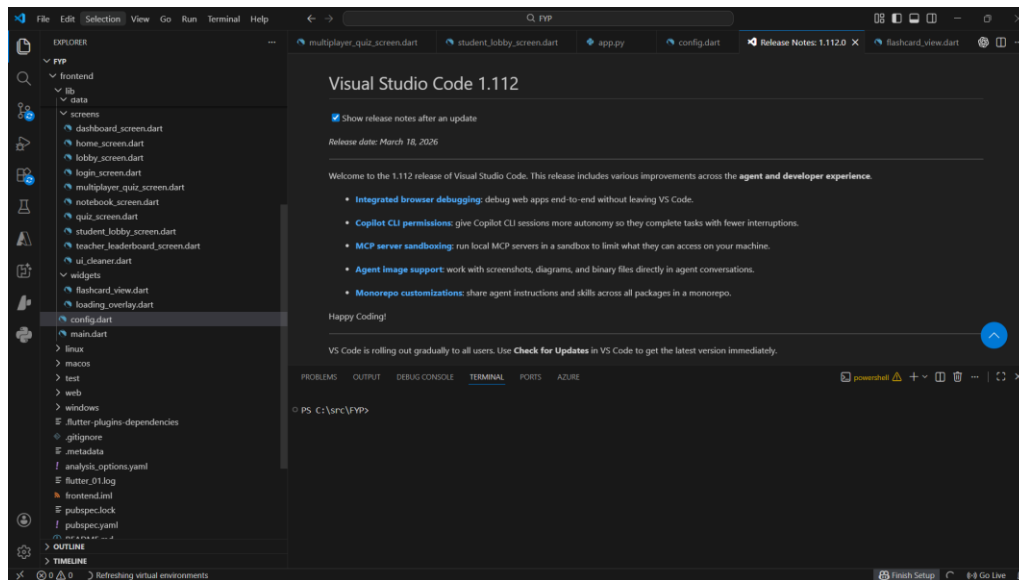


Figure 3.2.1 Visual Studio Code development environment

Figure 3.2.1 shows the Visual Studio Code environment used to develop and debug the Flutter frontend and Flask backend.

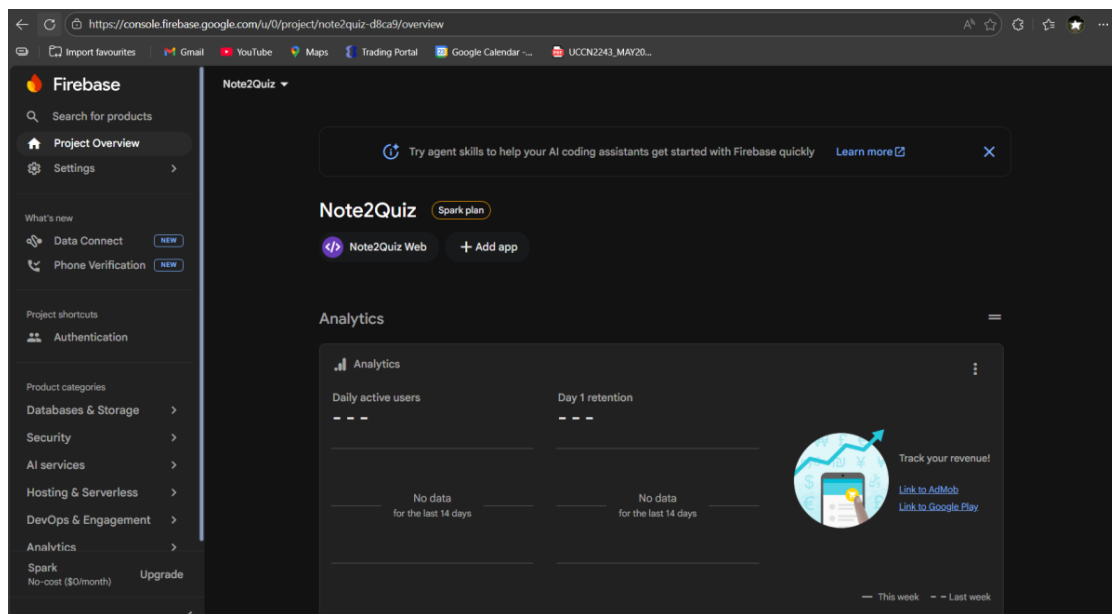


Figure 3.2.2 Firebase authentication configuration dashboard

Figure 3.2.2 shows the Firebase project dashboard used to configure Google-based sign-in for the web application.

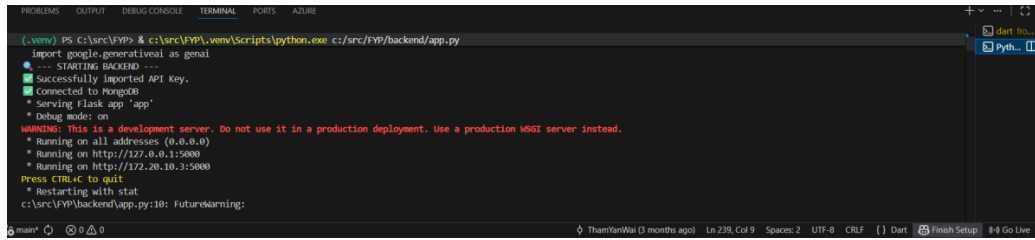


Figure 3.2.3 Flask backend runtime environment

Figure 3.2.3 shows the Flask backend running locally during development and API testing.

Collection name	Properties	Storage size	Documents	Avg. document size	Indexes	Total index size
activity_logs	-	32.77 kB	52	167.00 B	1	32.77 kB
notebooks	-	122.88 kB	4	18.51 kB	1	32.77 kB
users	-	32.77 kB	5	115.00 B	1	32.77 kB

Figure 3.2.4 MongoDB database environment

Figure 3.2.4 shows the MongoDB collections used to store notebooks, users, and activity logs.

3.3 System Design

3.3.1 System Architecture

Note2Quiz follows a client-server architecture. The user interacts with a Flutter frontend running in the browser, and the frontend communicates with a Flask backend through Representational State Transfer (REST) API calls. The backend coordinates notebook storage, source processing, AI generation, and multiplayer state. Firebase Authentication is used for Google sign-in, MongoDB stores persistent notebook and activity data when available, and the Gemini API is used for AI-assisted generation. Prototype multiplayer state is kept in memory on the server side.

Figure 3.3.1 shows both request and reply directions so that the architecture reflects the full interaction path rather than only the outbound request path.

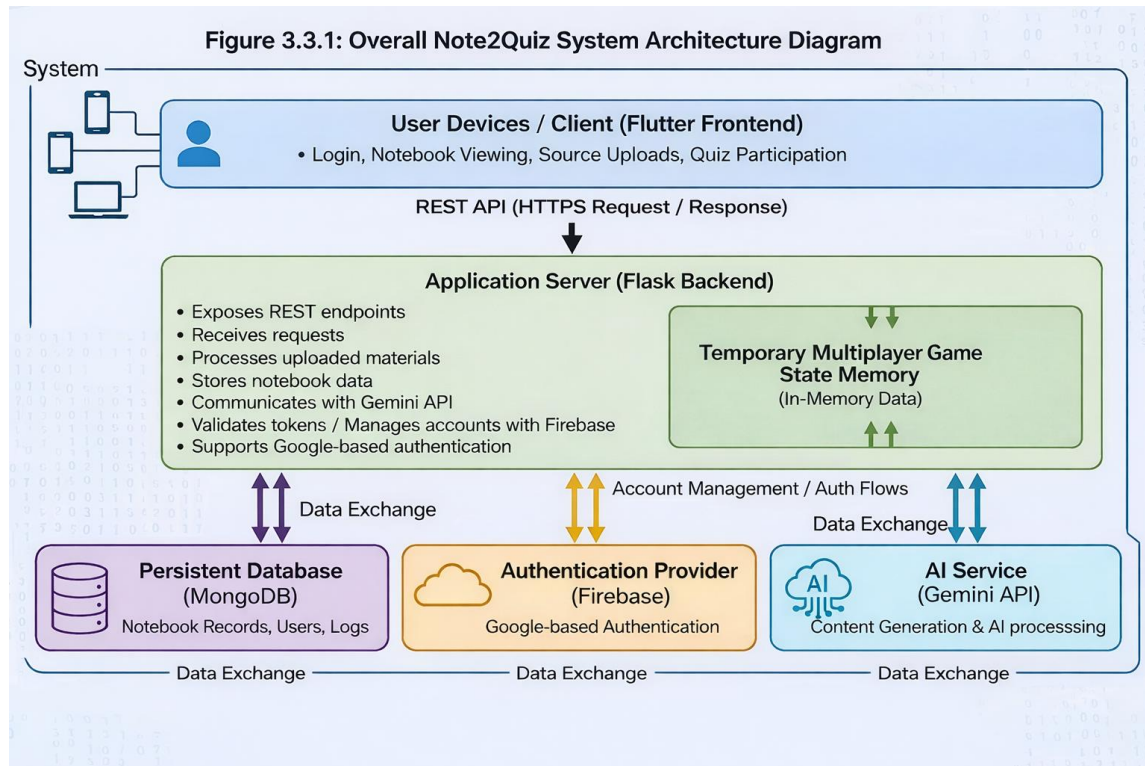
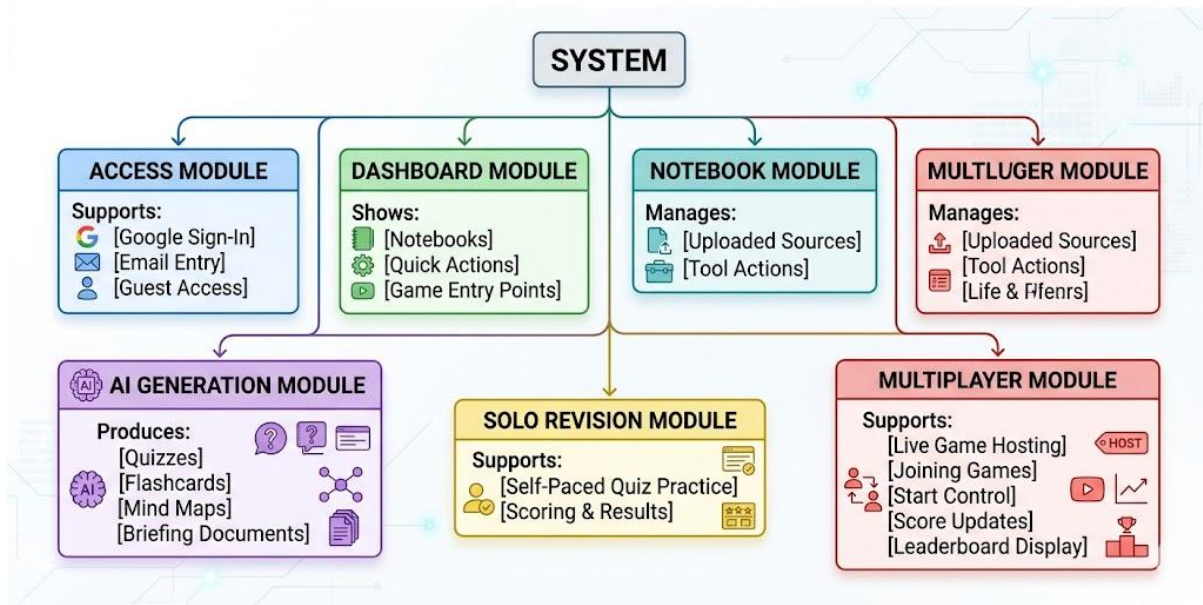


Figure 3.3.1 Overall system architecture of Note2Quiz

3.3.2 Module Design

The system is divided into modules so that each stage of the revision workflow has a clear responsibility. The Access Module handles sign-in and user identification before the Dashboard Module becomes available. The Dashboard Module provides entry to notebook workspaces and live-game participation. The Notebook Module stores the selected source context, while the AI Generation Module transforms that context into study output. The Solo Revision and Multiplayer Modules consume generated quiz content for different study scenarios.

Figure 3.3.2 presents the interaction between these modules. The diagram emphasizes that the modules are connected stages in a single workflow rather than independent features.

Figure 3.3.2: Top-down module design diagram**Figure 3.3.2** Top-down module interaction diagram

3.3.3 Data Flow Overview

The main data flow begins when the user accesses the system and creates a notebook to organize study materials. The user then uploads a document source, such as a PDF or PPTX file, through the frontend interface. Upon receiving the file, the backend processes the input and performs text extraction, converting the document into raw textual content. This extracted text is then stored within the corresponding notebook record in the database, forming the core context for subsequent operations.

After the content is prepared, the user selects a study action, which determines the type of output to be generated. At this stage, the backend aggregates all stored notebook sources and constructs a prompt context. This context is sent to the Gemini API, which generates different types of study content depending on the selected action. These outputs may include flashcards, notes, mind maps, or quiz questions. The generated content is then returned to the frontend either as structured data (e.g., quiz JSON format) or unstructured text (e.g., summaries or mind maps), and may also be stored in their respective data stores such as the flashcard store or notes store.

CHAPTER 3

For quiz-related functionality, the generated quiz data is passed into the quiz module, where the user chooses between solo or multiplayer modes. In solo mode, the quiz is presented directly to the user, and results are stored locally for revision purposes. In multiplayer mode, the system generates a unique game room code, allowing other users to join the session. Player responses and score updates are continuously synchronized with the backend, where the multiplayer game state is maintained. The results are then displayed through a leaderboard interface to support interactive learning.

Overall, this data flow demonstrates how Note2Quiz integrates document processing, AI-based content generation, and interactive quiz features into a unified workflow. The system enables a seamless transition from raw study materials to both individual revision and collaborative learning experiences.

However, the proposed data flow relies heavily on the quality of extracted text and the consistency of AI-generated responses. Variations in document structure, incomplete extraction, or ambiguous source content may affect the accuracy and usefulness of generated study materials. In addition, the dependence on external AI services introduces potential latency and reliability considerations, which may impact system responsiveness during peak usage.

Figure 3.3.3 illustrates this complete end-to-end process, from document upload and text extraction to AI-driven content generation and final usage in both solo and multiplayer revision modes

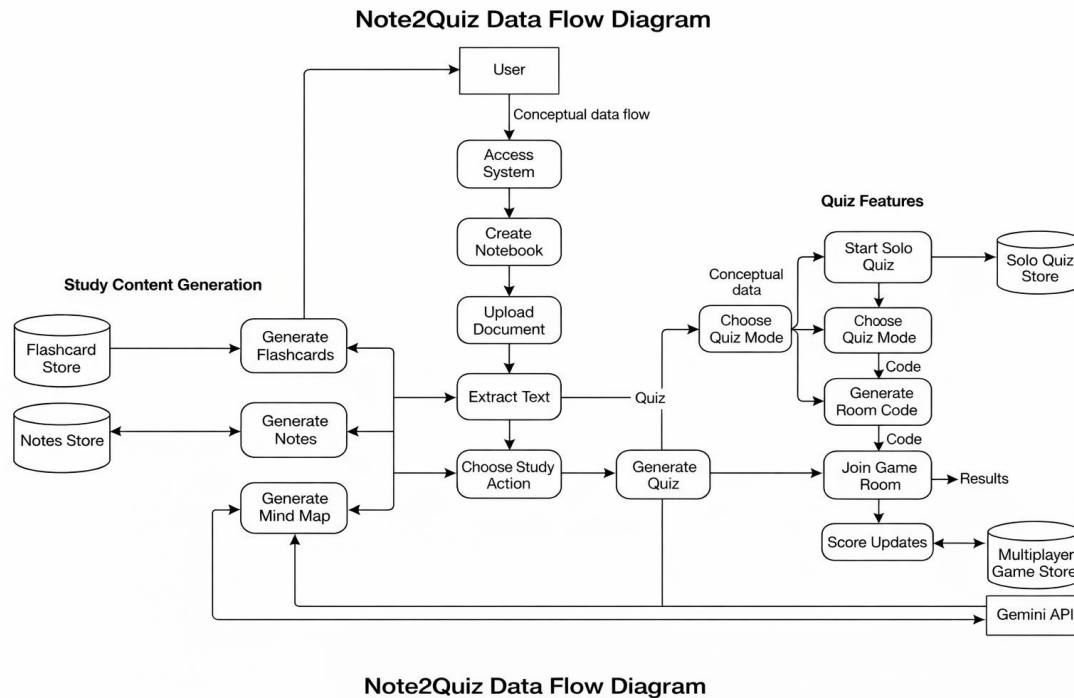


Figure 3.3.3 Data flow from notebook source upload to AI-generated study output

3.4 Functional Requirements

The functional requirements describe the specific behaviour that the system must support in order to meet the project objectives. The requirements below reflect the current implemented prototype and distinguish between access, notebook handling, content generation, revision, multiplayer interaction, and logging.

Table 3.3 Functional requirements of Note2Quiz (Part 1)

ID	Functional requirement	Description
FR1	User authentication	The system shall allow users to enter the application through Google sign-in, email-based access, or guest access.
FR2	Create notebook	The system shall allow a user to create a new notebook associated with a user identifier.
FR3	View notebook list	The system shall retrieve and display the notebooks stored for the active user.
FR4	Delete notebook	The system shall allow a user to delete an existing notebook from the dashboard.
FR5	Upload source	The system shall allow PDF and PPTX study materials to be uploaded into a

		notebook.
FR6	Extract and store source text	The backend shall extract readable text from the uploaded source and store it inside the corresponding notebook for AI processing.

Table 3.4 Functional requirements of Note2Quiz (Part 2)

ID	Functional requirement	Description
FR7	Generate quiz set	The system shall generate one quiz set from notebook content according to the selected difficulty level.
FR8	Generate flashcards	The system shall generate flashcards from notebook content.
FR9	Generate summary-style text output	The system shall generate mind-map style summaries and briefing-style report text from notebook content.

Table 3.5 Functional requirements of Note2Quiz (Part 3)

ID	Functional requirement	Description
FR10	Solo quiz interaction	The system shall present quiz questions, accept answers, and display score-based results for self-revision.
FR11	Host live game	The system shall allow the host to create a live quiz session and generate a unique game code.
FR12	Join live game	The system shall allow a player to join a valid live game code with a nickname before the game starts.
FR13	Leaderboard update	The system shall update and display player scores during a multiplayer session.
FR14	Activity logging	The backend shall record selected actions such as account registration, login, notebook creation, source upload, and game hosting.

3.5 Project Milestone

The project milestones are divided into Project I and Project II so that planning, implementation, refinement, and evaluation activities can be staged clearly across the FYP period. The charts below present a week-based milestone view that can be aligned later with the official semester calendar if required by the final submission template.

3.5.1 Project I Timeline

Project I focuses on problem analysis, literature review, interface and architecture planning, core backend development, source upload support, AI integration, and the initial prototype of solo and multiplayer revision flows. Figure 3.5.1 summarizes the major activities across the Project I timeline.

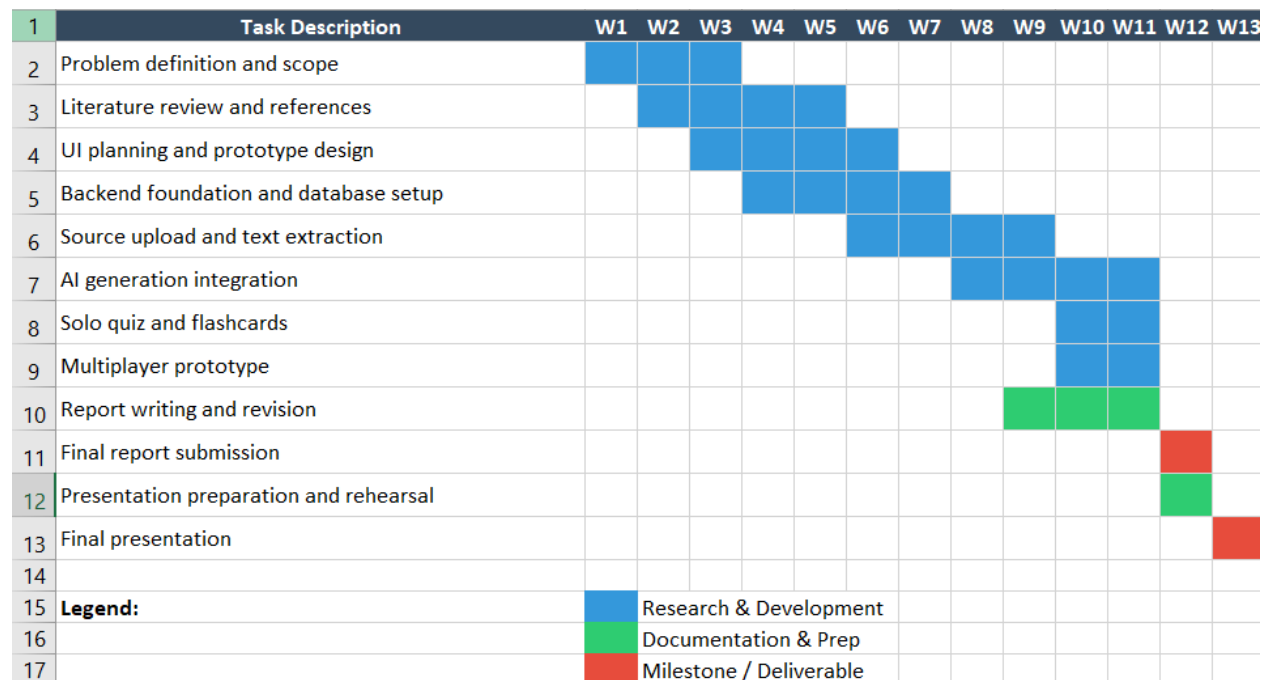


Figure 3.5.1 Project I milestone chart

3.5.2 Project II Timeline

Project II is expected to focus on module refinement, full integration testing, user evaluation, analysis of findings, improvement of the prototype based on feedback, and final report completion. Figure 3.5.2 provides the corresponding milestone plan for the next stage of the project.

CHAPTER 3

1	Task Description	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13
2	System refinement and bug fixing													
3	UI/UX improvement and user feedback integration													
4	Performance optimization and backend enhancement													
5	AI feature expansion (Generate PPT slides from notes)													
6	AI response tuning and prompt optimization													
7	Multiplayer feature finalization and stability testing													
8	Web deployment setup (hosting on public website)													
9	Database and API deployment (cloud integration)													
10	Integration testing (frontend, backend, AI, deployment)													
11	System testing and debugging (functional + usability)													
12	User testing and feedback collection													
13	Final system refinement and optimization													
14	Documentation and report writing													
15	Final system deployment and validation													
16	Final report submission													
17	Presentation preparation and rehearsal													
18	Final presentation													
19														
20	Legend:													
21														
22														

Figure 3.5.2 Project II milestone chart

3.6 Concluding Remark

This chapter presented the methodology used to develop Note2Quiz. After comparing common development models, the Prototype model was selected because it best matched the project's evolving interface and workflow requirements. The chapter also defined the system requirements, overall architecture, module interaction, data flow, functional requirements, and milestone plan that guide the development of the prototype.

Chapter 4

Preliminary Work

4.1 Software Setup and Configuration

This section describes how the software environment was prepared for the current prototype. In line with the chapter guideline, the discussion focuses on installation, configuration, and the integration of essential services rather than on feature behaviour. The main software setup tasks involved preparing the Flutter frontend environment, configuring the Python and Flask backend environment, integrating Firebase Authentication, preparing MongoDB collections, and defining the frontend-to-backend connection address.

4.1.1 Frontend Development Environment

The frontend was developed with Flutter and Dart in Visual Studio Code. Figure 4.1.1 shows the project opened in the development environment. At this stage, the frontend setup included the Flutter SDK, the Dart SDK, package dependencies for HTTP requests, file selection, and Firebase-based authentication. This environment was necessary because the project relies on multiple stateful screens, asynchronous API calls, and browser-based testing of the Flutter interface.

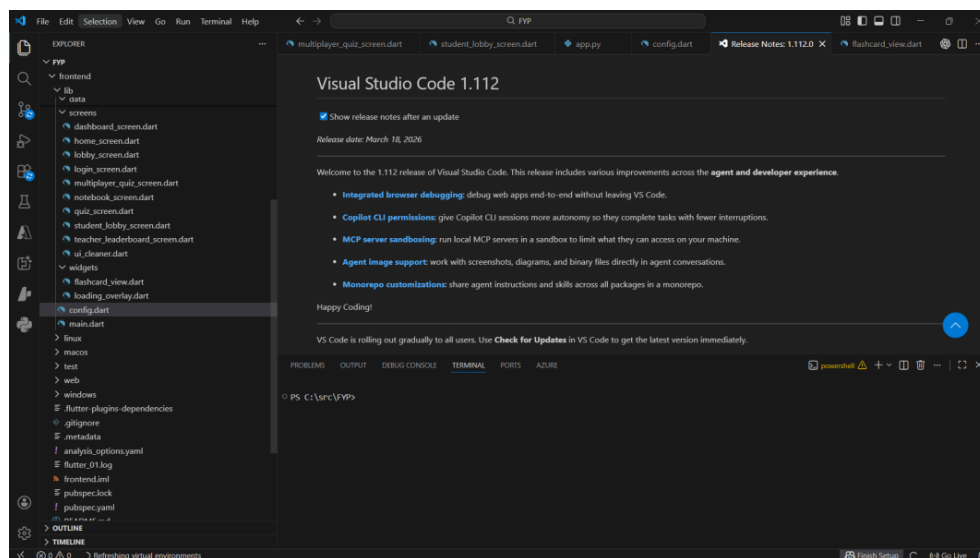


Figure 4.1.1 Visual Studio Code with the Note2Quiz project opened

4.1.2 Firebase Authentication Configuration

Firebase was configured to support the Google-based sign-in flow used by the application. The Firebase web project was created, the web app was registered, and the Firebase configuration values were placed in the Flutter entry point. Figure 4.1.2 shows the configured Firebase project dashboard. This configuration was required so that the Google sign-in action in the login screen could authenticate the user before the email identifier was sent to the backend login endpoint for notebook association and activity logging.

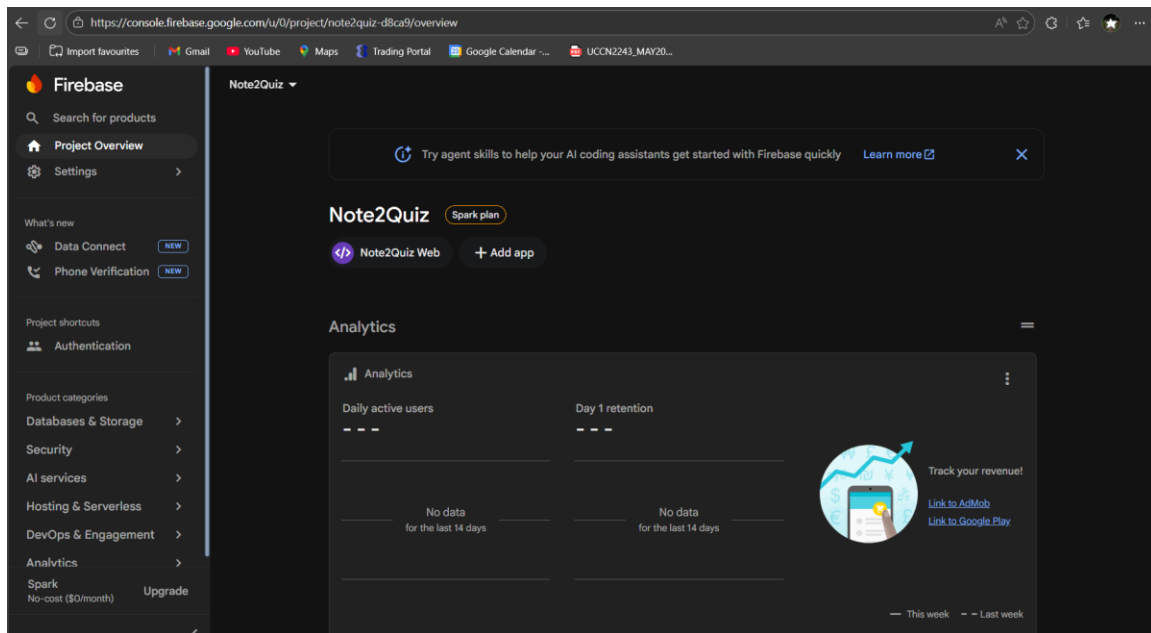
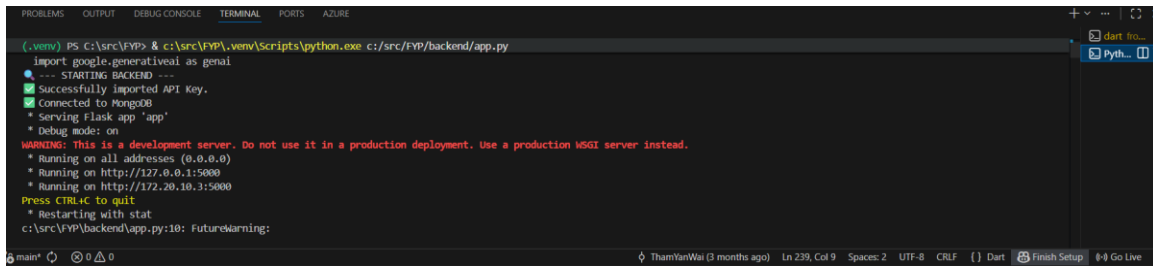


Figure 4.1.2 Firebase project dashboard used for web authentication setup

4.1.3 Backend Environment and Required Libraries

The backend environment was prepared with Python, Flask, and a set of supporting libraries listed in the project requirements file. These libraries include flask-cors for cross-origin communication, google-generativeai for Gemini access, pypdf for PDF parsing, python-pptx for PowerPoint parsing, youtube-transcript-api for future extensibility, and pymongo for MongoDB access. Figure 4.1.3 shows the Flask backend running locally. This setup was necessary so that the browser-based Flutter frontend could send API requests to a local server during development.



```

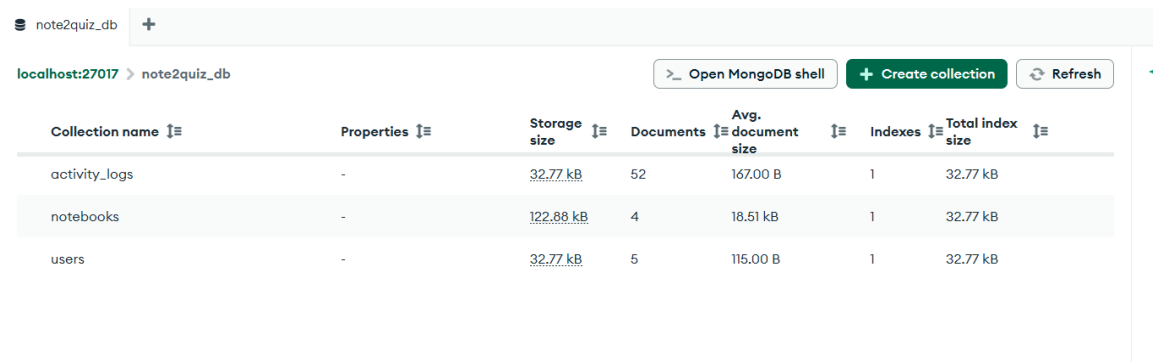
(.venv) PS C:\src\FYP> & c:\src\FYP\.venv\Scripts\python.exe c:\src\FYP\backend/app.py
import google.generativeai as genai
--- STARTING BACKEND ---
Successfully imported API Key.
Connected to MongoDB
Serving Flask app 'app'
Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.20.10.3:5000
Press CTRL+C to quit
* Restarting with stat
c:\src\FYP\backend\app.py:10: FutureWarning:

```

Figure 4.1.3 Flask backend running in the terminal during local development

4.1.4 Database Configuration

MongoDB was configured as the main persistent database for notebooks, users, and activity logs. The backend attempts to connect to the local MongoDB instance during startup and falls back to in-memory notebook storage when the service is unavailable. Figure 4.1.4 shows the MongoDB database collections used by the project. This configuration is important because it allows notebook data to persist across sessions while also preserving a development-time fallback path when MongoDB is unavailable.



Collection name	Properties	Storage size	Documents	Avg. document size	Indexes	Total index size
activity_logs	-	32.77 kB	52	167.00 B	1	32.77 kB
notebooks	-	122.88 kB	4	18.51 kB	1	32.77 kB
users	-	32.77 kB	5	115.00 B	1	32.77 kB

CHAPTER 4

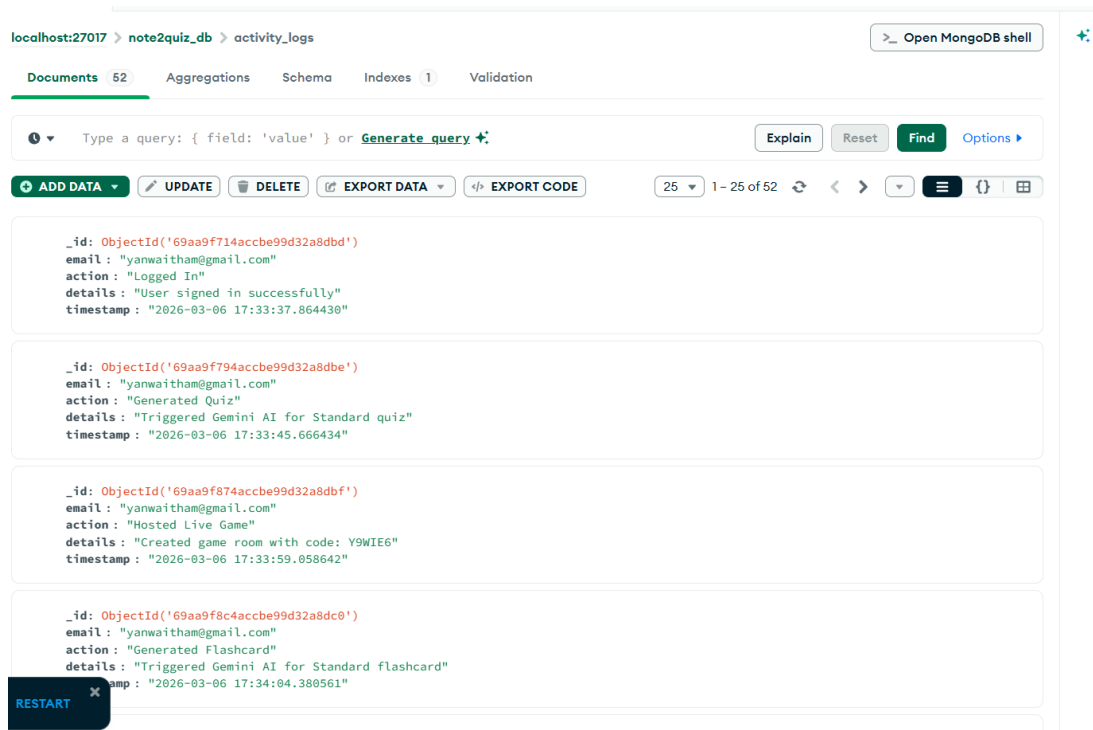


Figure 4.1.4 MongoDB collections used for notebooks, users, and activity logs

4.1.5 Frontend-to-Backend Address Configuration

A dedicated configuration file was used to centralize the backend base URL. The file selects 127.0.0.1 for Flutter web testing and 10.0.2.2 for Android emulator access. Figure 4.1.5 shows the base URL configuration. This step is necessary because a Flutter frontend running in different environments cannot always use the same loopback address to reach the backend server successfully.

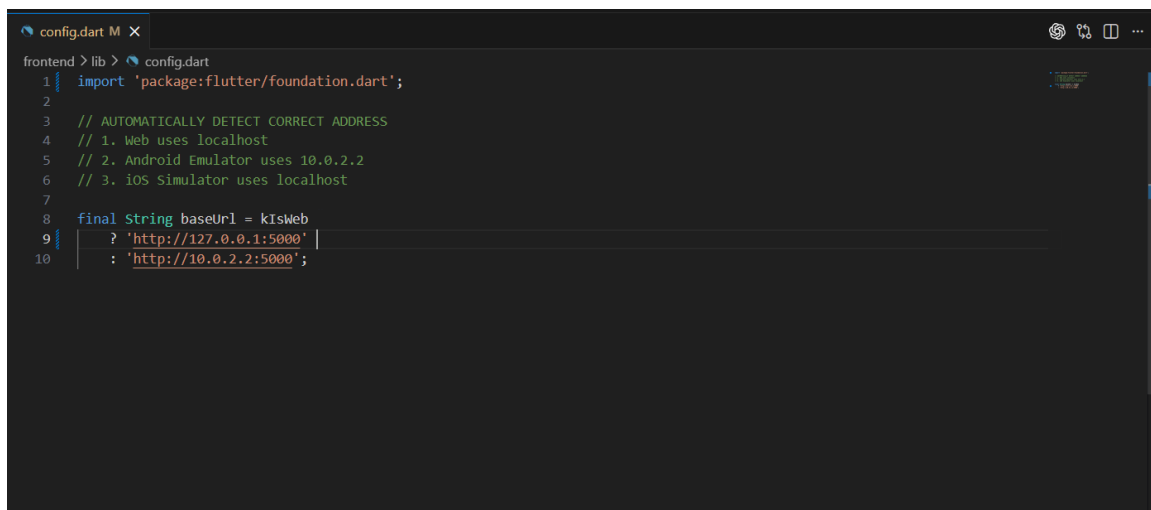


Figure 4.1.5 Base URL configuration for web and emulator environments

4.2 Implementation Details

This section explains how the major modules were implemented. Unlike the system-operation section, the focus here is on implementation logic, data handling, and selected code structures used to realise the required behaviour.

While the implementation successfully demonstrates the integration of multiple modules, it also reveals certain limitations in terms of scalability and maintainability. The current prototype prioritizes functional validation over optimization, and some components, such as in-memory multiplayer state handling and sequential API calls, may require further refinement for production-level deployment. These aspects will be addressed in the subsequent phase of the project.

4.2.1 Access and Login Module

The login module was implemented with three access paths: Google sign-in, direct email-based access, and guest access. For Google sign-in, the Flutter frontend uses Firebase Authentication to obtain the user identity. The resulting email address is then sent to the backend /login route so that the backend can register the user if necessary and log the sign-in action. For the email and guest paths, the frontend directly posts an email value to the same route. This design keeps the backend identity model simple because notebook ownership is consistently tied to an email identifier regardless of the access path used.

```
@app.route('/login', methods=['POST'])
def login():
    data = request.json
    email = data.get('email', '').strip().lower()

    if not email:
        return jsonify({"error": "Email is required"}), 400

    if USING_MONGO:
        existing_user = users_col.find_one({"email": email})

        if not existing_user:
            new_user = {
                "email": email,
                "created_at": str(datetime.datetime.now()),
                "role": "student"
            }
            users_col.insert_one(new_user)
            log_activity(email, "Registered Account", "First time user sign-in") # ✅ Logging Registration
        else:
            log_activity(email, "Logged In", "User signed in successfully") # ✅ Logging normal Login

    return jsonify({"success": True, "email": email, "message": "login successful"})
```

Listing 4.1 Core logic of the login module (frontend excerpt)

```

@app.route('/login', methods=['POST'])
def login():
    data = request.json
    email = data.get('email', '').strip().lower()

    if not email:
        return jsonify({"error": "Email is required"}), 400

    if USING_MONGO:
        existing_user = users_col.find_one({"email": email})

        if not existing_user:
            new_user = {
                "email": email,
                "created_at": str(datetime.datetime.now()),
                "role": "student"
            }
            users_col.insert_one(new_user)
            log_activity(email, "Registered Account", "First time user sign-in") # ✅ Logging Registration
        else:
            log_activity(email, "Logged In", "User signed in successfully") # ✅ Logging normal Login

    return jsonify({"success": True, "email": email, "message": "Login successful"})

```

Listing 4.2 Backend login route (backend excerpt)

4.2.2 Dashboard and Notebook Management Module

The dashboard module retrieves notebooks for the active user and displays them as selectable study spaces. When the user creates a notebook, the frontend sends a create request containing the current user email and notebook title. The backend responds by generating a unique notebook identifier, storing the notebook document, and returning the new notebook to the client. Deletion follows the reverse pattern. This implementation keeps notebook operations lightweight while making the dashboard the central navigation point for both notebook work and live-game entry.

4.2.3 Source Upload and Text Extraction Module

Source upload is implemented as a multipart file request from the Flutter notebook screen. The frontend file picker restricts the current interface to PDF and PPTX files. After upload, the backend inspects the filename extension and calls the relevant parsing function. Parsed text is then wrapped in a source object and appended to the notebook document. This logic makes the notebook a reusable context store for later generation requests.

```

if 'file' in request.files:
    f = request.files['file']
    title = f.filename
    if f.filename.endswith('.pdf'): content = extract_text_from_pdf(f)
    elif f.filename.endswith('.pptx'): content = extract_text_from_pptx(f)
    else:
        content = request.form.get('content', '')

    if not content.strip(): return jsonify({"error": "No text extracted"}), 400

    new_source = {
        "id": str(uuid.uuid4()),
        "title": title,
        "content": content,
        "type": source_type,
        "date": str(datetime.datetime.now())
    }

```

Listing 4.3 Source upload and extraction logic (backend excerpt)

4.2.4 AI Content Generation Module

The AI generation module is centred on the /generate-studio-item endpoint. When the frontend selects a tool type, the backend retrieves the combined notebook context, constructs a prompt for the selected study output, and then calls the Gemini API through a fallback model list. Structured outputs such as quiz sets and flashcards are cleaned and parsed as JSON, while briefing-style reports and mind-map style summaries are returned as text. The current implementation therefore separates generation type, prompt design, and output post-processing in a straightforward way.

```

if tool_type == "quiz":
    prompt = f"""Generate a {difficulty} Quiz with exactly {num_questions} questions.
    STRICT JSON FORMAT:
    [
        {{
            "question": "Question text here?",
            "options": ["Option A", "Option B", "Option C", "Option D"],
            "answer": "Option A",
            "hint": "A short, helpful clue without giving the answer away."
        }}
    ]
    Context: {context[:35000]}"""

elif tool_type == "flashcard":
    prompt = f"""Generate 10 Flashcards. Format: [{"front": "Term", "back": "Definition"}]. Context: {context[:35000]}"""

elif tool_type == "mindmap":
    prompt = f"""Create a hierarchical Mind Map using Emojis (🌱, 🐛). No Markdown blocks. Context: {context[:35000]}"""

elif tool_type == "report":
    prompt = f"""Write an Executive Briefing Doc with bold headers. Context: {context[:35000]}"""

```

Listing 4.4 Prompt selection for AI generation (backend excerpt)

4.2.5 Solo Quiz and Flashcard Modules

The solo quiz and flashcard modules are implemented on the frontend as separate interaction views that consume generated data. The quiz screen maintains the current question index, checks selected answers against the correct option, and updates the score before showing the final result. The flashcard view stores the current card index and supports front-back flipping behaviour. These modules were intentionally kept independent of the generation endpoint so that the same generated dataset can be displayed in different ways without changing the backend logic.

4.2.6 Multiplayer Game Module

The multiplayer game module is implemented through a small group of backend routes that create a room code, accept player joins, expose game status, update player scores, and return leaderboard data. When a host launches a live game, the generated quiz data is sent to the /host-game route, which stores the session in the in-memory active_games structure. Students then join with a room code and nickname. The teacher leaderboard and student lobby periodically poll the backend for state changes. This approach was selected because it provides a stable and understandable prototype implementation without introducing an additional real-time socket stack at the current project stage.

```
@app.route('/host-game', methods=['POST'])
def host_game():
    data = request.json
    code = ''.join(random.choices(string.ascii_uppercase + string.digits, k=6))
    email = data.get('email', 'guest')

    active_games[code] = {
        "host_email": email,
        "quiz_data": data.get('quiz_data', []),
        "players": {},
        "status": "waiting"
    }

    log_activity(email, "Hosted Live Game", f"Created game room with code: {code}")
    return jsonify({"code": code})
```

```

@app.route('/join-game', methods=['POST'])
def join_game():
    data = request.json
    code = data.get('code', '').upper()
    name = data.get('name', 'Anonymous')

    if code in active_games:
        if active_games[code]['status'] != 'waiting':
            return jsonify({"error": "Game already started! Too late to join."}), 400

        active_games[code]['players'][name] = 0
        log_activity("player_join", "Joined Game", f"Player '{name}' joined room {code}")
        return jsonify({
            "success": True,
            "quiz_data": active_games[code]['quiz_data']
        })
    return jsonify({"error": "Invalid Game Code"}), 404

```

Listing 4.5 Multiplayer room creation and joining logic (backend excerpt)

4.3 System Operation

4.3.1 Design Layout

The current prototype includes a distinct layout for each major user-facing module. The layout decisions were intended to keep the revision workflow visually clear and to reduce unnecessary navigation between steps.

Figure 4.3.1 shows the landing page design, which introduces the application and presents the high-level value proposition before the user proceeds to the login flow.

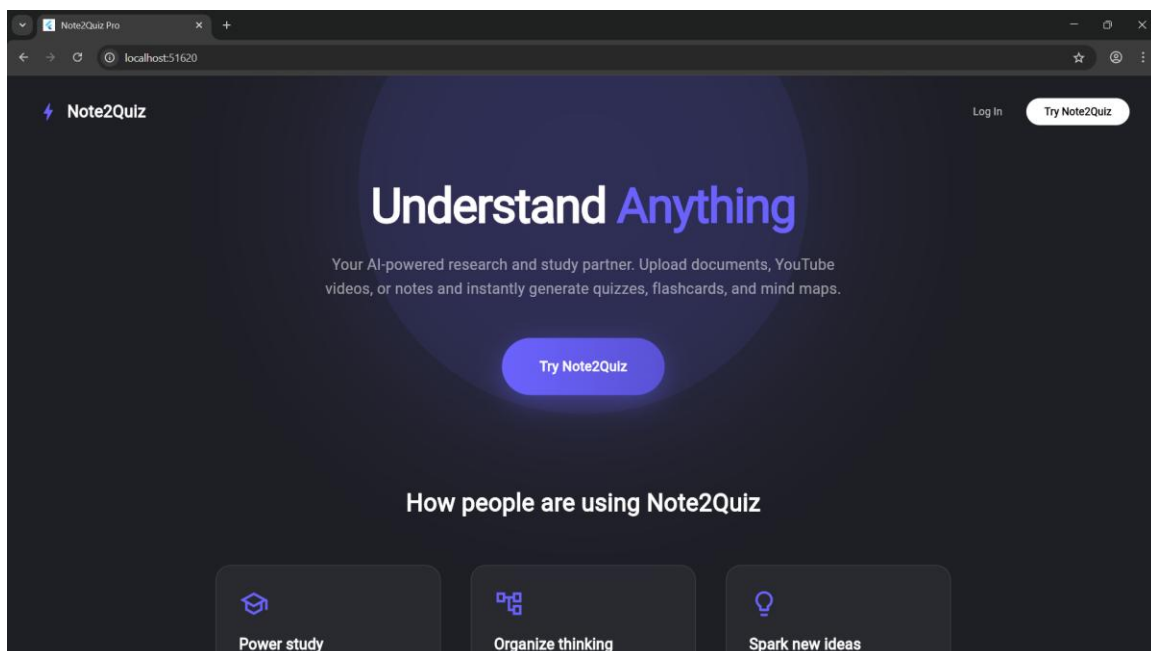


Figure 4.3.1 Landing page design

Figure 4.3.2 shows the login page design. The page groups Google sign-in, a direct email path, and a guest-access option inside one card so that entry choices remain simple and visible.

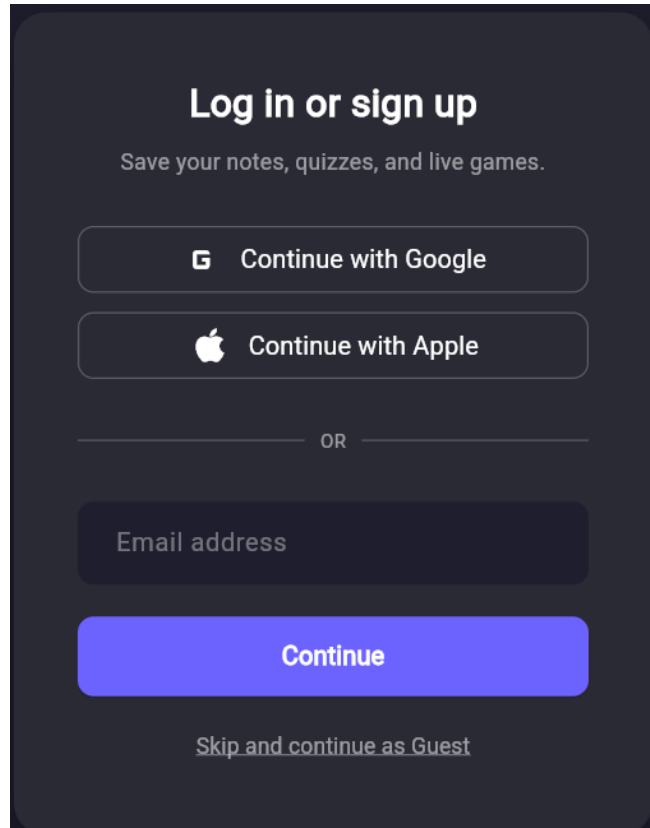


Figure 4.3.2 Login page design

Figure 4.3.3 shows the dashboard layout. The dashboard acts as the navigation hub by displaying notebook cards, quick actions, and access to live-game participation.

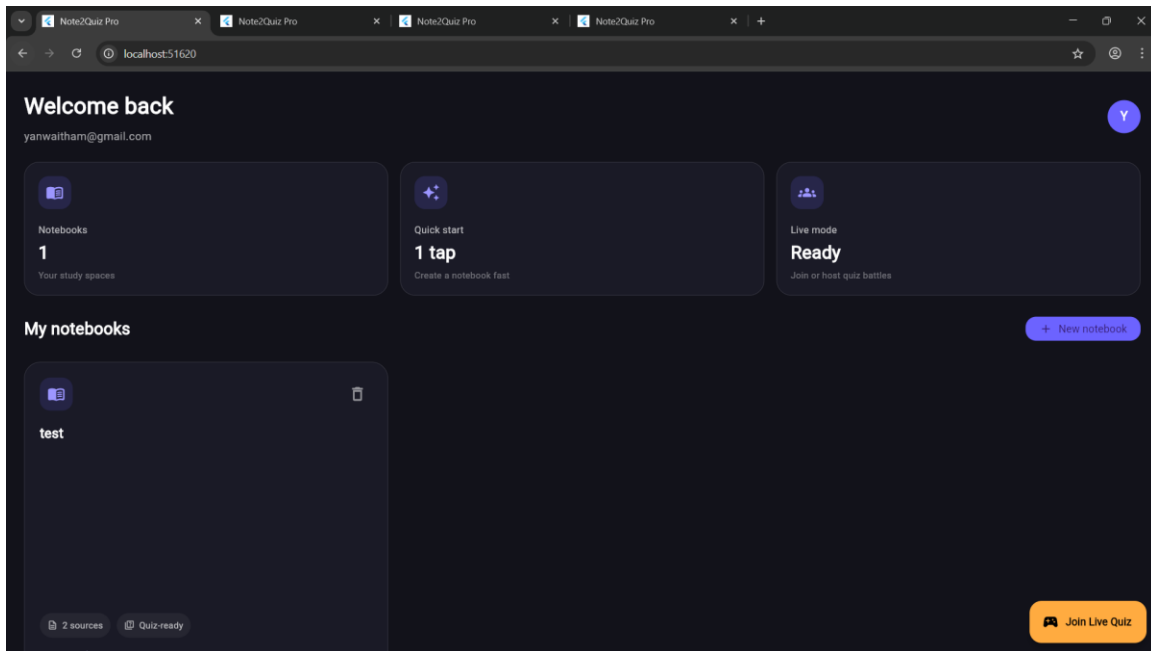


Figure 4.3.3 Dashboard design

Figure 4.3.4 presents the notebook workspace design. The screen separates sources, main content, and study actions so that upload and generation can happen without leaving the notebook context.

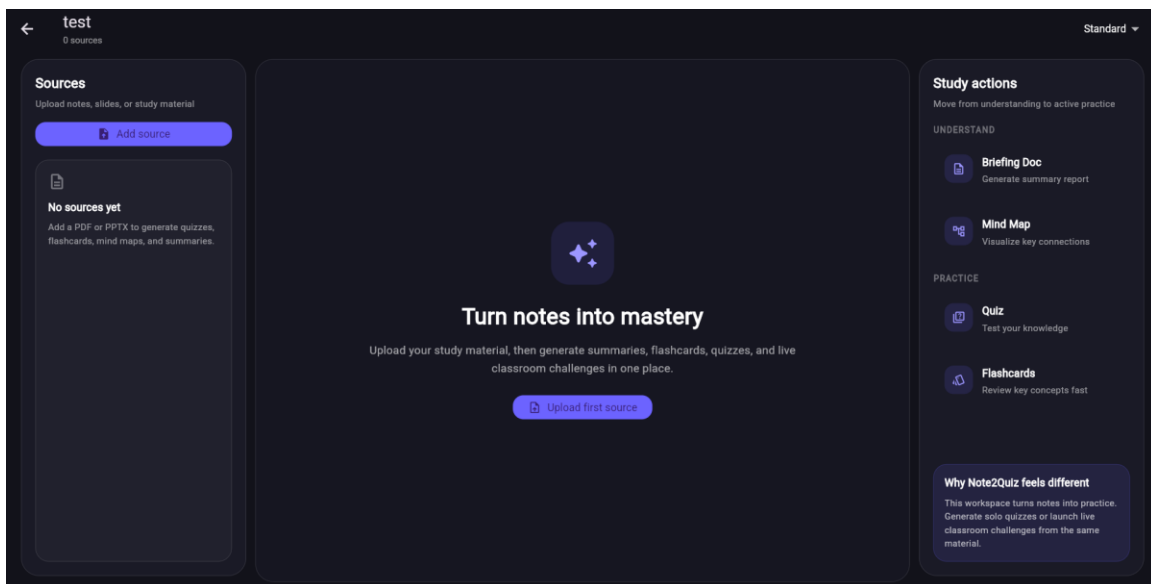


Figure 4.3.4 Notebook workspace design

Figure 4.3.5 shows the solo quiz design, which emphasizes one question at a time, clear answer choices, and a visible score/progress area for focused revision.

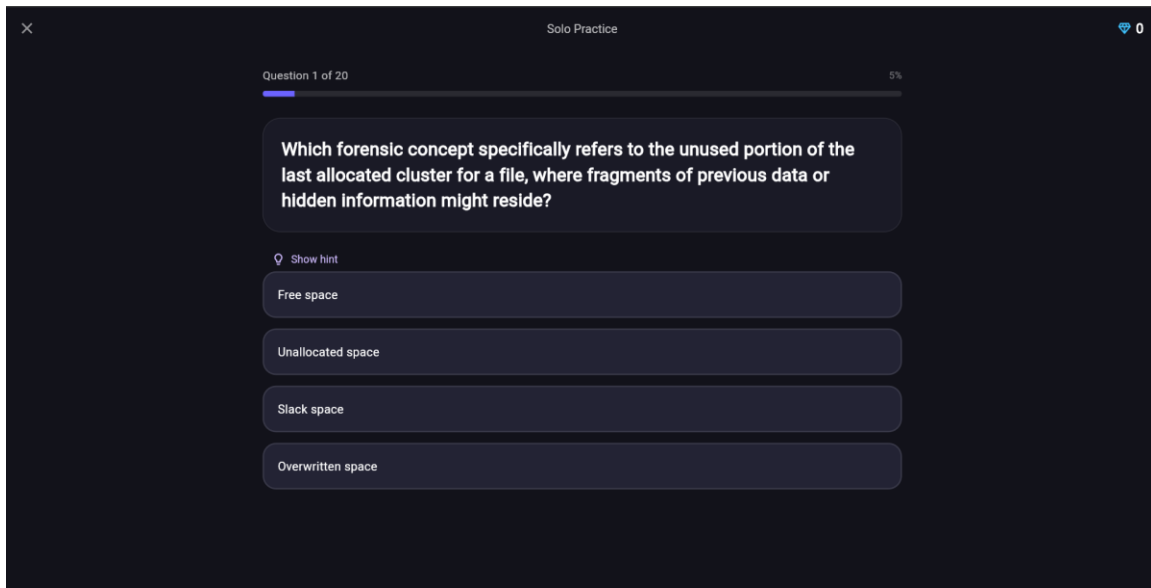


Figure 4.3.5 Solo quiz design

Figure 4.3.6 shows the student lobby layout used before the multiplayer quiz begins. The screen provides a simple waiting state that displays the joined room code.

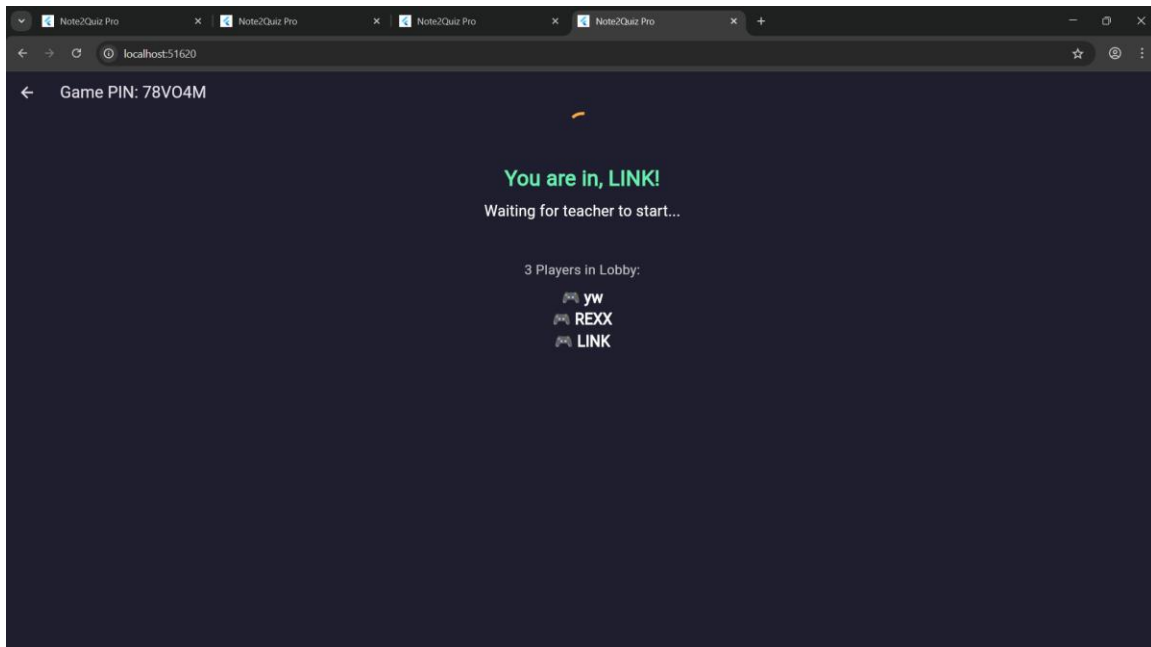


Figure 4.3.6 Student lobby design

Figure 4.3.7 shows the teacher leaderboard layout. The host can view the room code, monitor player participation, and start the live quiz session.

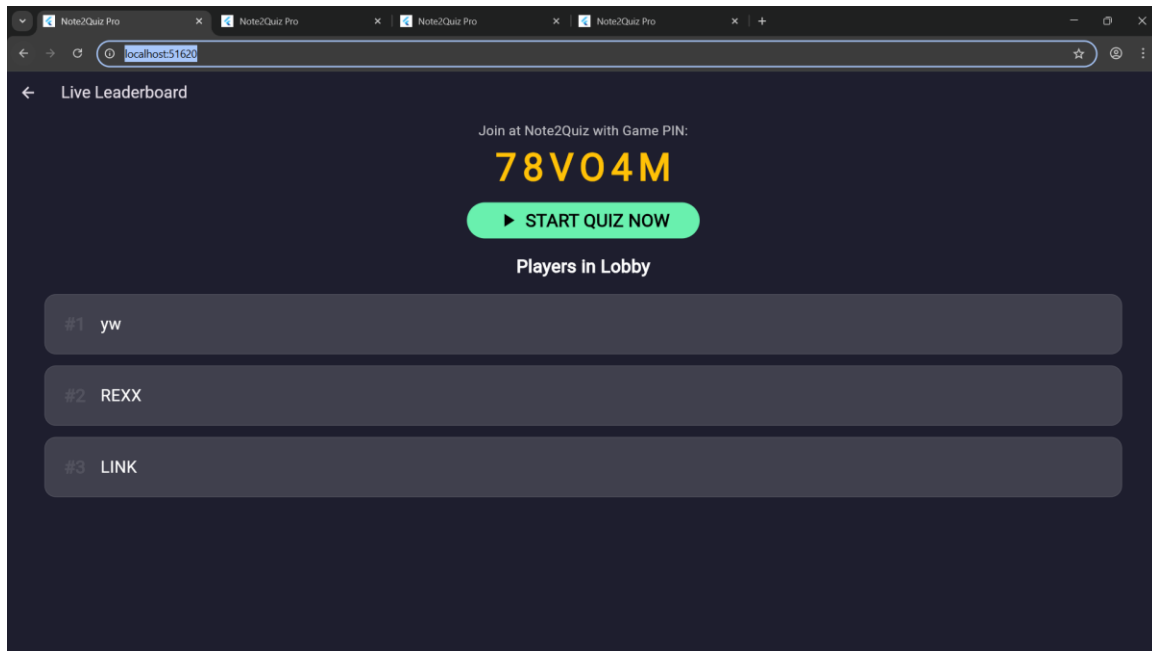


Figure 4.3.7 Teacher leaderboard design

4.3.2 Functional Operation

The functional operation of the prototype can be described as a sequence that begins with access and moves through notebook setup, source processing, content generation, and revision use.

The first operational step is notebook creation from the dashboard. Figure 4.3.8 shows the create-notebook dialog used to initialise a new study workspace for the active user.

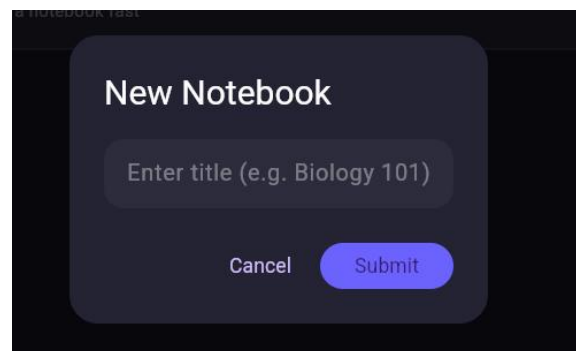


Figure 4.3.8 Notebook creation dialog

After a notebook has been created, the user uploads source material. Figure 4.3.9 shows the source upload operation where the file picker is used to select a PDF or PPTX file for backend processing.

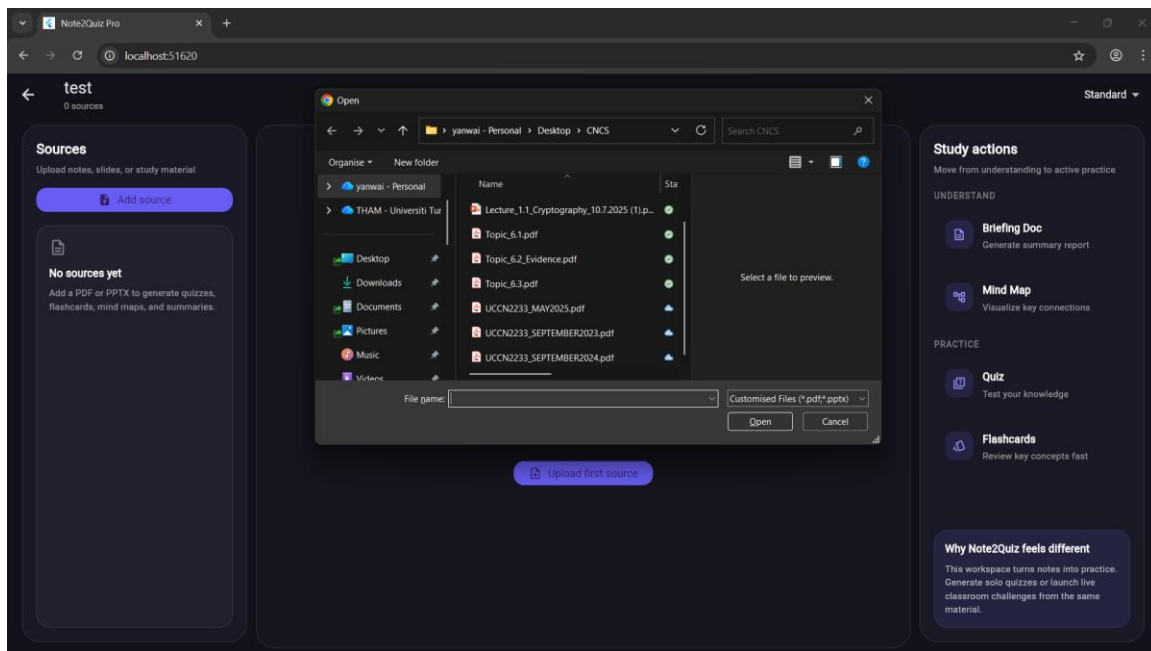


Figure 4.3.9 Source upload into a notebook

Once source text is stored, the user may trigger summary-style output. Figure 4.3.10 shows the generated briefing-style report text, while Figure 4.3.11 shows a generated mind-map style summary. These outputs demonstrate how the same notebook context can produce different study views.

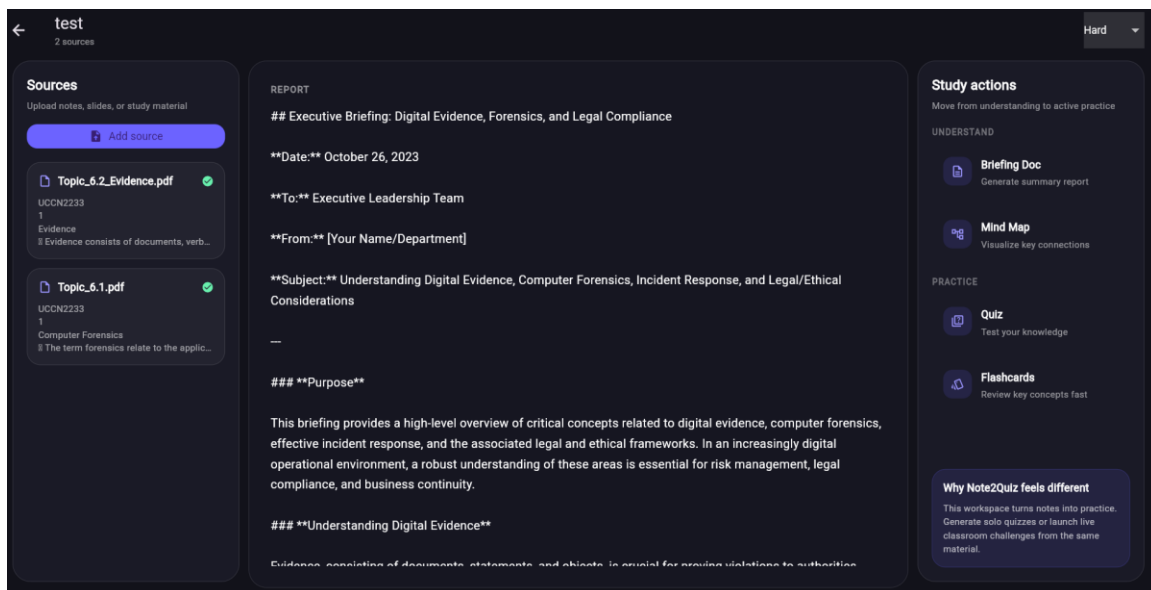


Figure 4.3.10 Generated briefing-style report output

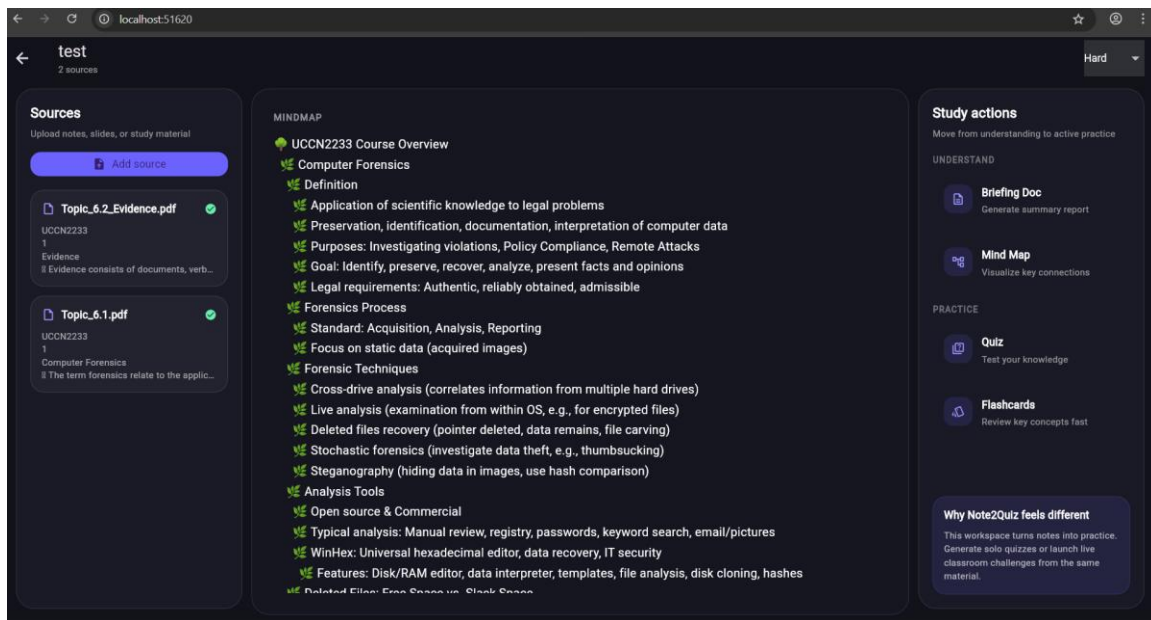


Figure 4.3.11 Generated mind-map style summary output

Quiz generation is presented through a ready state before interaction begins. Figure 4.3.12 shows the quiz-ready state, and Figure 4.3.13 shows the solo quiz screen after the user enters the question flow.

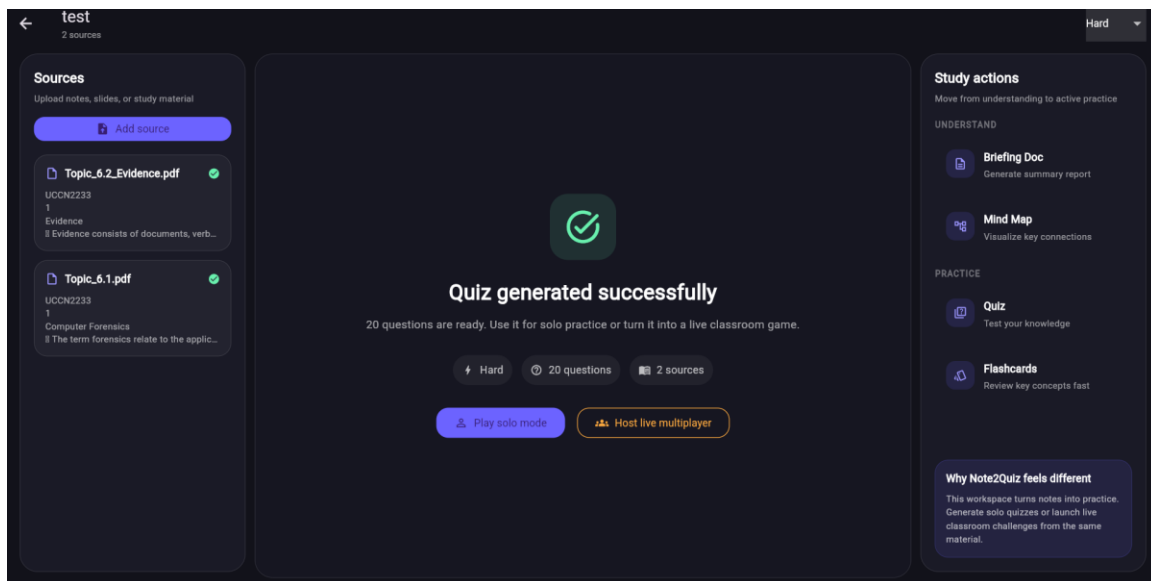


Figure 4.3.12 Quiz-ready state after generation

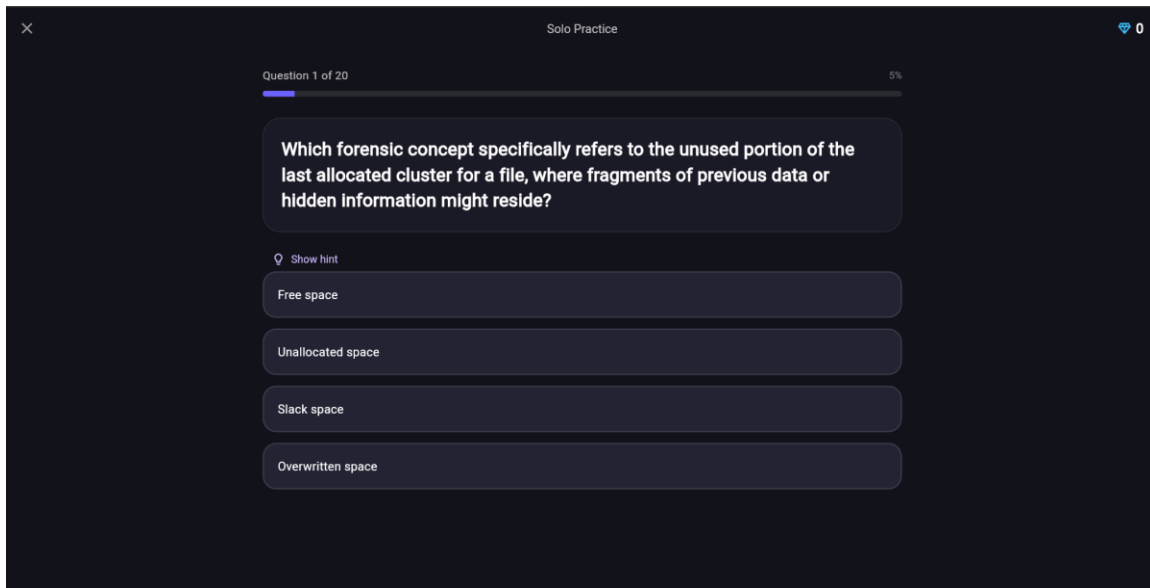


Figure 4.3.13 Solo quiz interaction screen

The flashcard operation uses the same notebook context in a faster memorization-oriented interaction mode. Figure 4.3.14 shows the flashcard interface used to flip between front and back content.

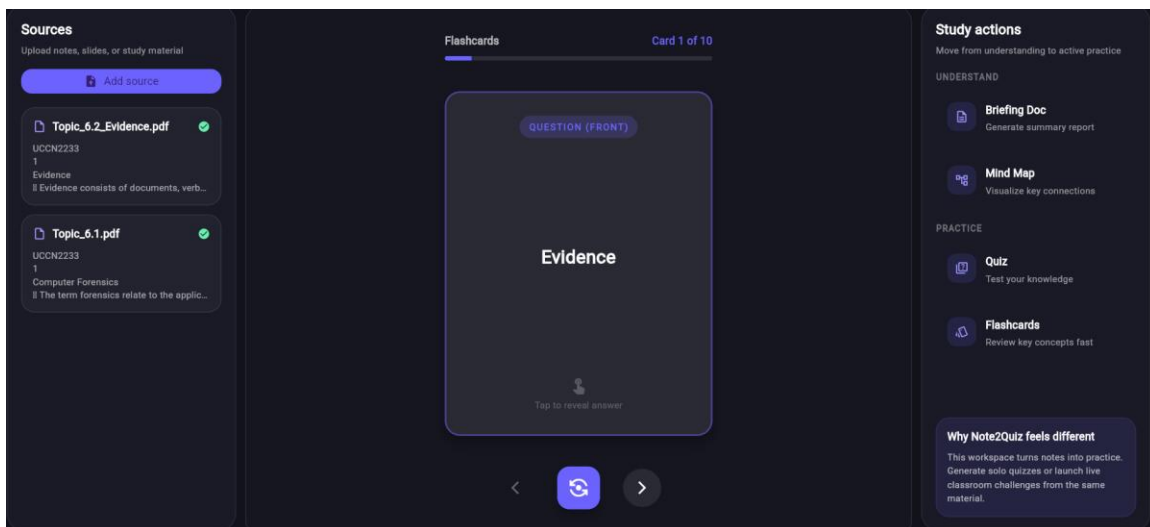


Figure 4.3.14 Flashcard interaction view

For multiplayer use, the host first creates a game room and the student joins through a code-based dialog. Figure 4.3.15 shows the join-live-game dialog, Figure 4.3.16 shows the teacher leaderboard before the session starts, and Figure 4.3.17 shows the live multiplayer quiz once the game is in progress.

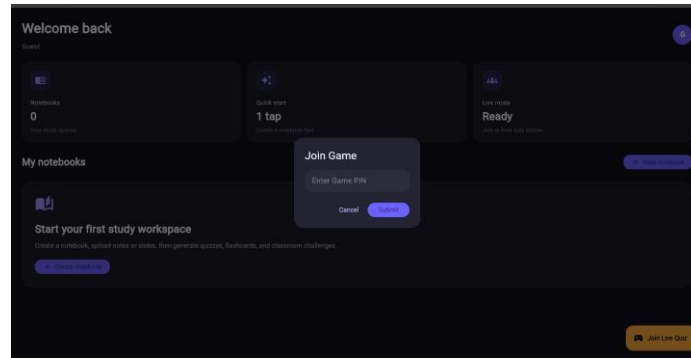


Figure 4.3.15 Join live game dialog

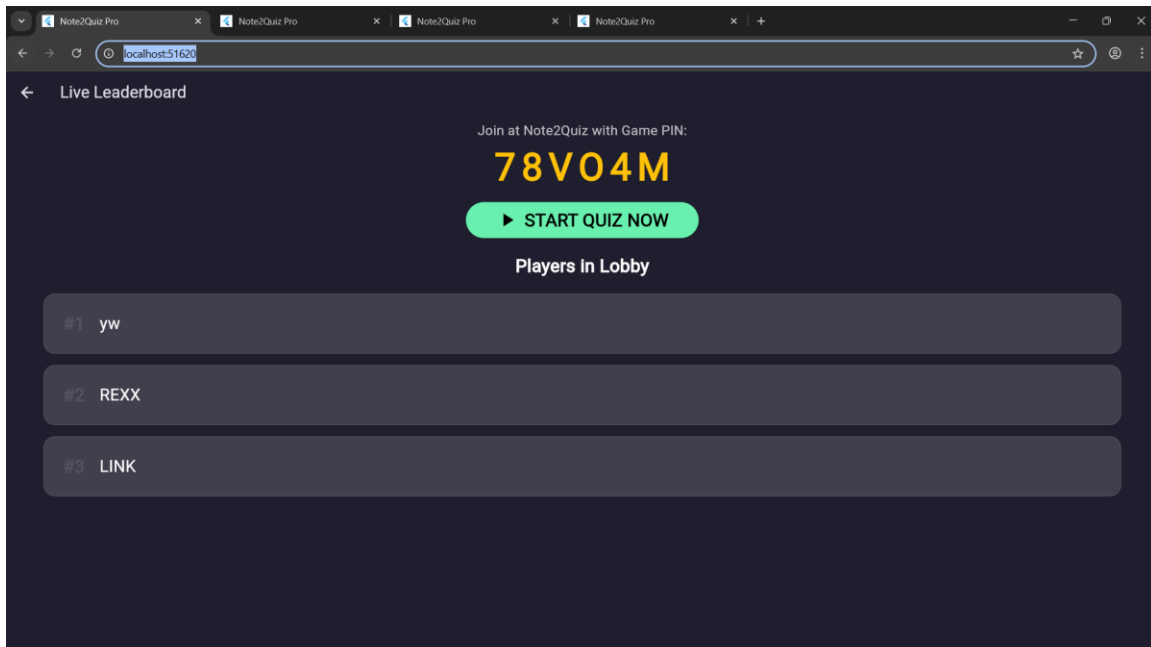


Figure 4.3.16 Teacher leaderboard with room code and start control

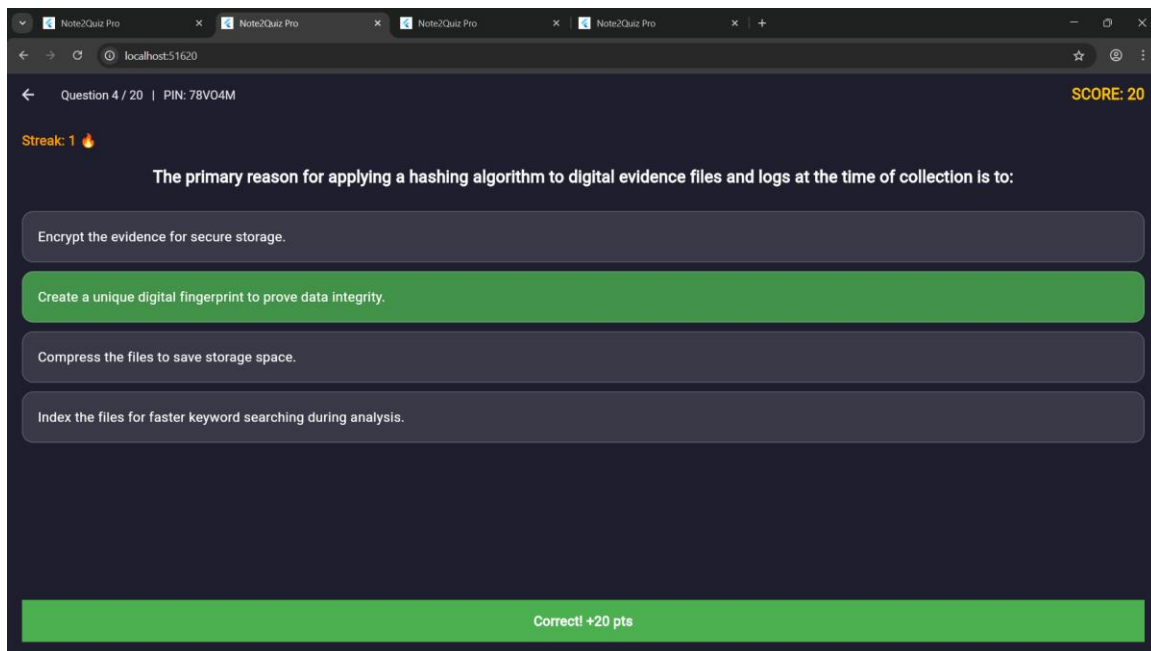


Figure 4.3.17 Multiplayer quiz gameplay screen

4.4 Implementation Issues and Challenges

One challenge was ensuring reliable AI output for structured generation. Quiz and flashcard modes require predictable JSON-like responses, but generative models may return extra text or formatting variations. To address this issue, the backend includes response-cleaning logic and a fallback model sequence. This improves robustness by giving the system more than one model option and by attempting to recover structured data when the initial response is not directly parseable.

A second challenge was source-text quality. PDF and PowerPoint files do not always preserve content in a clean reading order, especially when the original file contains decorative layouts or sparse slide text. To reduce this problem, the current prototype limits upload formats to PDF and PPTX in the main interface, uses dedicated extraction functions for each format, and stores the extracted content directly so it can be reviewed again if generation quality needs to be checked.

A third challenge was multiplayer synchronization. Real-time socket communication would provide more immediate updates, but it would also introduce additional infrastructure and debugging overhead for the current prototype stage. The selected polling approach was retained

CHAPTER 4

because it is easier to verify, sufficiently stable for small-room demonstrations, and adequate for validating the multiplayer concept before more advanced real-time work is attempted in FYP 2.

A fourth challenge was environment consistency across development tools and services. The system depends on a browser-based Flutter frontend, a local Flask backend, Firebase configuration, Gemini access, and MongoDB availability. Differences in loopback addresses, local service availability, and API key configuration can break the end-to-end workflow. To manage this issue, the project uses a centralized base URL file, backend fallback behaviour when MongoDB is unavailable, and explicit startup configuration for external services.

4.5 Concluding Remark

This chapter described the preliminary work completed for Note2Quiz by presenting the software setup, implementation logic, system operation, and key development challenges. The chapter shows that the project has already progressed from conceptual design to a working prototype in which notebooks, source upload, AI generation, solo quiz practice, and prototype multiplayer interaction operate as an integrated workflow.

CHAPTER 5

Conclusion

This report presented the FYP 1 stage of Note2Quiz, an AI-assisted study application designed to reduce the manual effort required to convert lecture notes into usable revision resources. The work completed so far demonstrates that the core prototype architecture is in place: a Flutter frontend provides user access, dashboard navigation, notebook interaction, and revision interfaces, while a Flask backend supports source upload, source text extraction, notebook management, AI generation, and prototype multiplayer coordination. The current implementation already supports Google sign-in, email-based or guest access, notebook creation, PDF and PPTX source upload, quiz and flashcard generation, briefing-style and mind-map style text generation, solo quiz interaction, and code-based multiplayer room creation and joining.

At the same time, the FYP 1 stage should be interpreted as a functional prototype rather than a fully evaluated final system. The project has shown that the proposed workflow is technically achievable, but formal usability testing, learning-effectiveness evaluation, and broader refinement remain part of the planned FYP 2 work. The prototype also still has known limitations: multiplayer state is stored in memory, the current interface does not expose every possible source type, and AI output quality still depends on source clarity and prompt behaviour. Even with these limitations, the present result establishes a strong technical foundation for the next stage of the project. In FYP 2, the work can be extended through deeper testing, user evaluation, interface refinement, and stronger real-time or deployment capabilities so that the prototype can be assessed more rigorously as a revision support system.

In conclusion, all three project objectives have been successfully achieved at the prototype level. The system demonstrates functional support for user access, notebook management, source upload, AI-assisted content generation, and both solo and multiplayer quiz interaction. These outcomes confirm that the proposed workflow—from raw study material to interactive revision output—is technically feasible within a unified system. Moving forward, the next phase of the project will focus on enhancing system robustness, improving AI output consistency, refining user experience through usability evaluation, and extending deployment

CHAPTER 5

readiness. These improvements are expected to strengthen the overall effectiveness of Note2Quiz as a practical and scalable revision support solution.

REFERENCES

References

- [1] H. L. Roediger and J. D. Karpicke, "Test-enhanced learning: Taking memory tests improves long-term retention," *Psychological Science*, vol. 17, no. 3, pp. 249–255, 2006.
- [2] H. L. Roediger and J. D. Karpicke, "The power of testing memory: Basic research and implications for educational practice," *Perspectives on Psychological Science*, vol. 1, no. 3, pp. 181–210, 2006.
- [3] E. Ratinho and C. Martins, "The role of gamified learning strategies in student motivation in high school and higher education: A systematic review," *Heliyon*, vol. 9, no. 8, 2023.
- [4] F. Pelizzari, "Gamification in higher education: A systematic literature review," *Italian Journal of Educational Technology*, 2024.
- [5] N. Khan and S. Saunderson, "Generative artificial intelligence in education: A narrative literature review," *World Journal on Educational Technology: Current Issues*, vol. 16, no. 2, pp. 112–126, 2024.
- [6] X. Wang, H. L. Chin, and Z. Zainuddin, "The effectiveness of generative AI in education: A systematic review of empirical study," *JuKu: Jurnal Kurikulum & Pengajaran Asia Pasifik*, vol. 12, no. 4, pp. 48–66, 2024.
- [7] Flutter, "Desktop support for Flutter," *Flutter documentation*. [Online]. Available: <https://docs.flutter.dev/platform-integration/desktop>. [Accessed: Apr. 16, 2026].
- [8] Meta, "Introduction," *React Native documentation*. [Online]. Available: <https://reactnative.dev/docs/getting-started>. [Accessed: Apr. 16, 2026].
- [9] Google, "Application fundamentals," *Android Developers*. [Online]. Available: <https://developer.android.com/guide/topics/fundamentals>. [Accessed: Apr. 16, 2026].
- [10] Apple, "Develop for iOS," *Apple Developer*. [Online]. Available: <https://developer.apple.com/ios/>. [Accessed: Apr. 16, 2026].
- [11] Pallets, "Flask Documentation," *Flask documentation*. [Online]. Available: <https://flask.palletsprojects.com/>. [Accessed: Apr. 16, 2026].
- [12] Django Software Foundation, "Django documentation," *Django documentation*. [Online]. Available: <https://docs.djangoproject.com/>. [Accessed: Apr. 16, 2026].
- [13] Express.js, "Using middleware," *Express documentation*. [Online]. Available: <https://expressjs.com/en/guide/using-middleware.html>. [Accessed: Apr. 16, 2026].
- [14] MongoDB, "MongoDB Documentation," *MongoDB documentation*. [Online]. Available: <https://www.mongodb.com/docs/>. [Accessed: Apr. 16, 2026].

REFERENCES

- [15] Oracle, "MySQL Documentation," *MySQL documentation*. [Online]. Available: <https://dev.mysql.com/doc/>. [Accessed: Apr. 16, 2026].
- [16] Google, "Cloud Firestore," *Firebase documentation*. [Online]. Available: <https://firebase.google.com/docs/firestore>. [Accessed: Apr. 16, 2026].
- [17] Google, "Gemini API reference," *Google AI for Developers*. [Online]. Available: <https://ai.google.dev/api>. [Accessed: Apr. 16, 2026].
- [18] Quizlet, "AI-Powered Tools: Practice Tests, Study Guides, Flashcards & More," *Quizlet*. [Online]. Available: <https://quizlet.com/features/ai-study-tools>. [Accessed: Apr. 16, 2026].
- [19] Kahoot!, "What is Kahoot!," *Kahoot!*. [Online]. Available: <https://kahoot.com/what-is-kahoot/>. [Accessed: Apr. 16, 2026].
- [20] Quizizz, "Live Session Modes on Quizizz," *Quizizz Help Center*. [Online]. Available: <https://support.quizizz.com/hc/en-us/articles/360030685632-Live-Session-Modes-on-Quizizz>. [Accessed: Apr. 16, 2026].
- [21] Google, "NotebookLM," *Google*. [Online]. Available: <https://notebooklm.google/>. [Accessed: Apr. 16, 2026].
- [22] Google, "Firebase Authentication," *Firebase documentation*. [Online]. Available: <https://firebase.google.com/docs/auth>. [Accessed: Apr. 16, 2026].
- [23] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. New York, NY, USA: McGraw-Hill, 2019.
- [24] I. Sommerville, *Software Engineering*, 10th ed. Harlow, U.K.: Pearson, 2015.

APPENDIX A

Supplementary Technical Notes

This appendix provides concise supplementary notes that support the technical discussion in Chapters 3 and 4. The material is included to strengthen documentation of the FYP I prototype while keeping the main chapters focused on the proposal narrative.

A.1 Representative Backend Route Summary

- **/login:** Accepts an email-based identifier, registers the user when necessary, and records access activity so that notebook ownership can be tied consistently to one user identity.
- **/generate-studio-item:** Receives the selected study tool request, aggregates notebook source context, and returns generated quiz, flashcard, mind-map style, or briefing-style output.
- **/host-game:** Creates a prototype multiplayer session by storing generated quiz data in server memory and returning a room code for classroom-style participation.
- **Additional notebook, upload, join, status, and leaderboard routes:** Support notebook creation and retrieval, source upload and text extraction, player entry into a room, state polling, score updates, and leaderboard display within the implemented prototype.

A.2 Illustrative Prototype Data Structures

Representative examples of the data handled by the prototype are shown below for documentation purposes. The examples are illustrative rather than exhaustive.

Representative notebook record

```
{
  "notebookId": "nb001",
  "userEmail": "student@example.com",
  "title": "Software Engineering Revision",
  "sources": [
    {
      "sourceTitle": "lecture01.pdf",
      "sourceType": "pdf",
      "extractedText": "...",
      "uploadedAt": "2026-04-16T10:30:00"
```

APPENDIX

```
    }  
  ]  
}
```

Representative quiz item

```
{  
  "question": "What is the main purpose of retrieval practice?",  
  "options": [  
    "Passive rereading",  
    "Long-term retention",  
    "File compression",  
    "User-interface styling"  
  ],  
  "answer": "Long-term retention"  
}
```

A.3 Planned Extension Items for FYP II

1. Strengthen validation and post-processing for structured AI outputs so that quiz and flashcard generation become more robust.
2. Improve multiplayer synchronization beyond the present polling approach when the project moves from prototype validation toward a more responsive live environment.
3. Conduct broader usability review, prototype evaluation, and refinement of interface flow based on findings collected during the next project stage.
4. Extend deployment readiness through stronger persistence, monitoring, and operational refinement once the core learning workflow has been validated.

APPENDIX

A.4 Poster



Note2Quiz

AI-Powered Learning Assistant for University Students

Transforming Study Notes into Summaries, Quizzes & Interactive Learning

Student Name : Tham Yan Wai

Student ID : 22ACB06270

Supervisor : Tan Lyk Yin

Email : 2206270@utar.my



Project Demo

1 INTRODUCTION

- Students spend too much time revising notes.
- Difficult to convert notes into effective study materials.
- Existing tools provide limited support.



2 PROBLEM STATEMENT

- Manual revision is time-consuming and repetitive.
- Hard to identify key concepts from notes.
- Limited tools for structured and interactive learning.

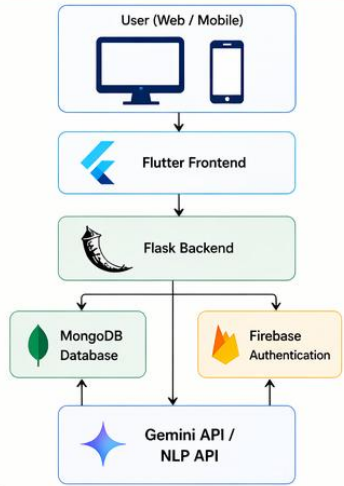
3 OBJECTIVES

- Develop a web-based learning assistant.
- Generate summaries and quizzes using AI.
- Improve revision efficiency and engagement.
- Support interactive learning (quiz, flashcards, mind map).

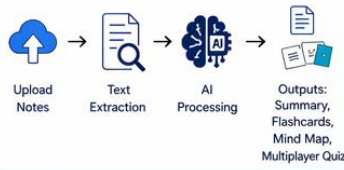
4 PROPOSED METHOD

- Upload Notes**
Upload notes or slides (PDF, PPT, etc.)
- Text Extraction**
Extract text content from uploaded files.
- AI Processing (Gemini API)**
AI understands content and extracts key information.
- Generate Outputs**
Generate different learning materials automatically.
- Student Revision**
Students use outputs to revise, practice and track progress.

5 SYSTEM ARCHITECTURE



PROCESS FLOW OVERVIEW



TECHNOLOGY STACK

- Frontend : Flutter
- Backend : Flask
- Database : MongoDB
- Authentication : Firebase Authentication
- AI Service : Gemini API

SAMPLE WORKFLOW (MOCKUP)

My Notebook + Upload New

Dashboard	Data Structures.pdf Uploaded on 12 May 2005 · 12 pages
Notebooks	Machine Learning.pptx Uploaded on 9 May 2005 · 28 slides
Uploads	Operating System.pdf Uploaded on 6 May 2005 · 15 pages

Generated Outputs

AI Summary
View summary

Flashcards
120 cards

Mind Map
View map

Multiplayer Quiz
Start quiz

AI SUMMARY PREVIEW (MOCKUP)

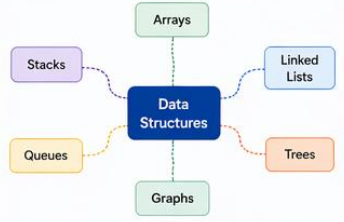
Summary

Data structures are used to organize and store data efficiently. Common types include arrays, linked lists, stacks, queues, trees and graphs. The choice of data structure affects the performance of algorithms in terms of time and space complexity.

6 CORE FEATURES

A MIND MAP

Automatically visualizes key concepts for better understanding.



B MULTIPLAYER QUIZ

Join live quiz rooms, compete with classmates and track real-time scores.

Join Room

Room Code

7 X 9 K 2

Join

Live Leaderboard

1	Alice	1200
2	Ben	980
3	Chloe	760
4	Daniel	650

Players (4 / 10)

Alice	1200
Ben	980
Chloe	760
Daniel (You)	650

Q3. Which data structure uses LIFO (Last In, First Out) principle?

A. Queue

B. Stack ✓

C. Tree

D. Graph

15s

C FLASHCARDS

Card-based revision for quick recall and spaced repetition.

1 / 20 ☆ ⋮

Q. What is a queue in data structures?

Click to reveal answer

D AI SUMMARY / QUIZ GENERATION

AI automatically generates summaries and quiz questions from your notes.



8 EXPECTED CONTRIBUTION

- Reduce revision time significantly.
- Improve learning efficiency.
- Increase engagement through interactive content.
- Support self-paced and classroom learning.

9 CONCLUSION

Note2Quiz transforms study notes into interactive learning tools using AI, improving revision efficiency and student engagement.



Keywords: AI-assisted learning • Text summarization • Question generation

Revision support • NLP • Educational technology

FYP | Proposal